



UNIVERSIDADE FEDERAL
DO ESPÍRITO SANTO

Centro Tecnológico
Departamento de Informática

Prof. Vítor E. Silva Souza

<http://www.inf.ufes.br/~vitorsouza>

Controle de Versões com Git



This material is licensed under the Creative Commons license Attribution-ShareAlike 4.0 International: <http://creativecommons.org/licenses/by-sa/4.0/>.

Crédito

Parte dos slides
deste curso foram
baseados em
material cedido
pelo prof. João
Paulo A. Almeida.



Gerenciamento de Configuração de Software

- *Configuration Management – CM;*
- Sistemas de software estão em constante mudança:
 - *Bugs, novos requisitos, novos hardwares/plataformas, etc.*
- Sistemas pensados como conjuntos de versões, cada uma deve ser mantida e gerenciada;
- Atividades:
 - *Gerenciamento de mudanças;*
 - *Gerenciamento de versões;*
 - *Construção do sistema;*
 - *Gerenciamento de releases.*



Mais sobre isso nas disciplinas de Engenharia de Software



Gerenciamento de versões

1) The amount of space to store the source code (whether on disk, tape or cards) may be several times that needed for any particular version. For example, there might be “customer,” “system test,” and “development” source libraries, with most modules represented by a different version in each.¹

2) Fixes made to one version of a module sometimes fail to get made to other versions.

3) When changes occur it is difficult to tell exactly what changed and when.

4) When a customer has a problem it is hard to figure out what version he has.

Marc Rochkind, The Source Code Control System, 1975
(construído no OS/360, para uso no Bell Labs, AT&T)



What Are the Benefits of Git?

Git has several key benefits:

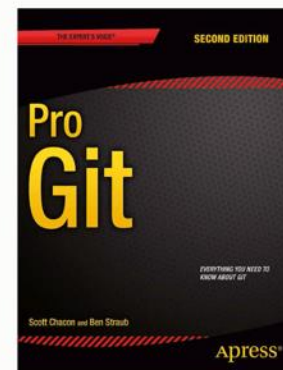
- **Historical Change Tracking** – You can review a graph of how your commits have changed over time, see when and by whom changes were made, and revert to a previous commit if needed. This history makes it easier to identify and fix bugs.
- **Work as a Team** – You can easily share your code with teammates for review before you commit or merge back to the main working branch. Additionally, the branching and review capabilities enable simultaneous development. Multiple people can work on the same file and resolve differences later.
- **Improve Team Speed & Productivity** – Git makes it easy for your team to track changes to your code. Now you can focus on writing code instead of spending time tracking and merging different versions across your team. Additionally, Git performs computations and stores your main repository locally, making it quicker on most operations than a centralized VCS.
- **Availability and Redundancy** – Git is a distributed VCS, meaning there is no single, central place where everything is stored. In a distributed system, there are multiple backups in the event that you need one. This approach also means that you can work offline and commit your changes when you're ready.
- **Git Is the Industry Standard** – Due to its popularity, Git is supported by many integrated development environments (IDEs) and many popular developer tools including [AWS CodeCommit](#), Jenkins, and Travis. There are many free Git resources available, such as the [Git open source](#) page.



Book

The entire Pro Git book, written by Scott Chacon and Ben Straub and published by Apress, is available here. All content is licensed under the [Creative Commons Attribution Non Commercial Share Alike 3.0 license](#). Print versions of the book are available on [Amazon.com](#).

The version found here has been updated with corrections and additions from [hundreds of contributors](#). If you see an error or have a suggestion, patches and issues are welcome in its [GitHub repository](#).



2nd Edition (2014)

1. Começando

- 1.1 Sobre Controle de Versão
- 1.2 Uma Breve História do Git
- 1.3 O Básico do Git

Download Ebook



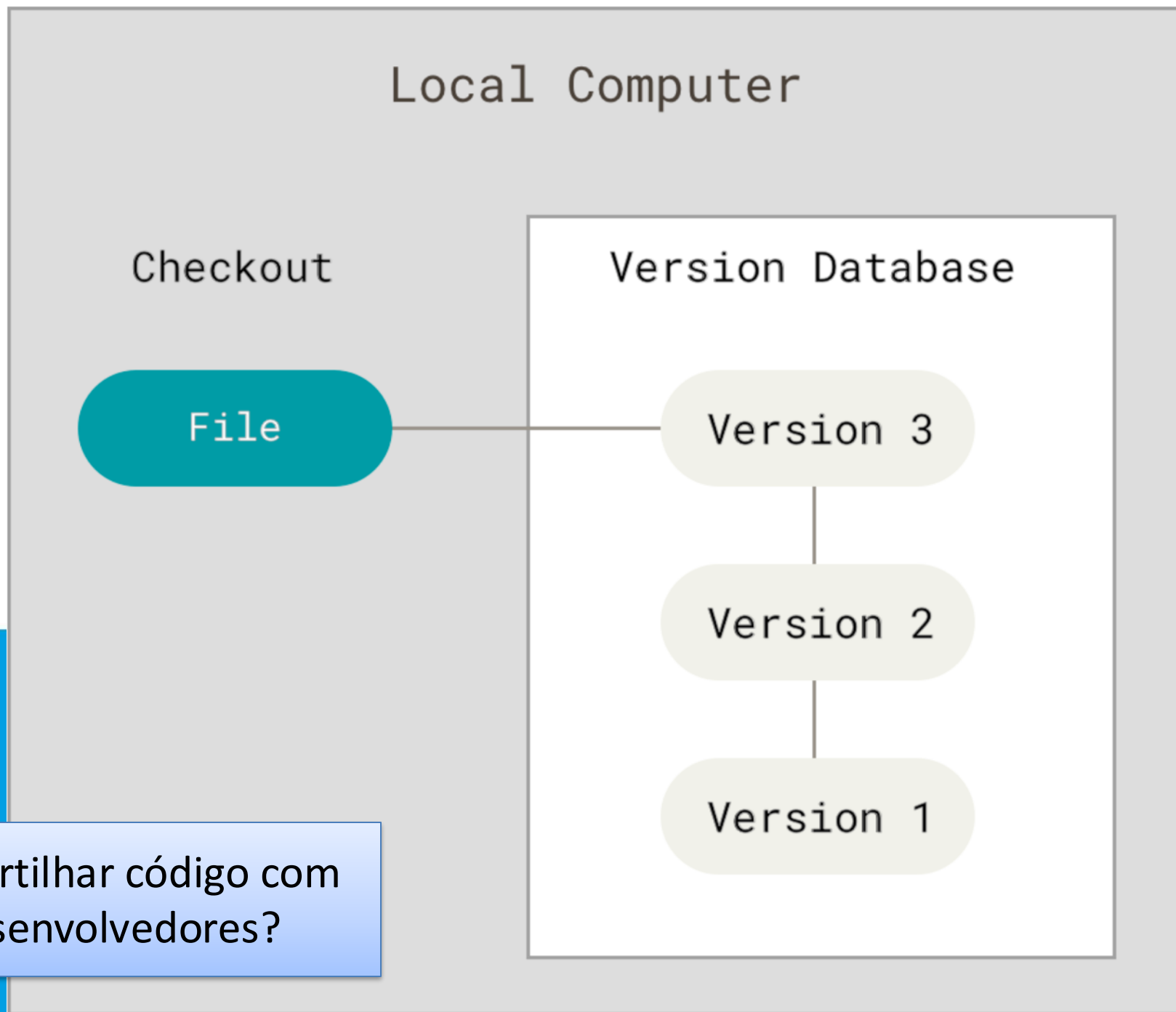
<https://git-scm.com/book/pt-br/v2>

Controle de Versão

- Registro de alterações em um arquivo ou conjunto de arquivos ao longo do tempo;
 - *Permite restaurar qualquer versão dos arquivos;*
 - *Nosso foco são artefatos de software, mas funciona com qualquer tipo de arquivo;*
- Sistema de Controle de Versão ou *Version Control System (VCS)*:
 - *Manuais ou automatizados;*
 - *Locais, centralizados ou distribuídos.*

VCS Local

Ex.: RCS



Como compartilhar código com
outros desenvolvedores?

VCS Centralizado

Ex.: CVS, Subversion, Perforce

Ponto único de falha...

```
graph TD; SR[shared repository] <--> D1[developer]; SR <--> D2[developer]; SR <--> D3[developer];
```

shared
repository

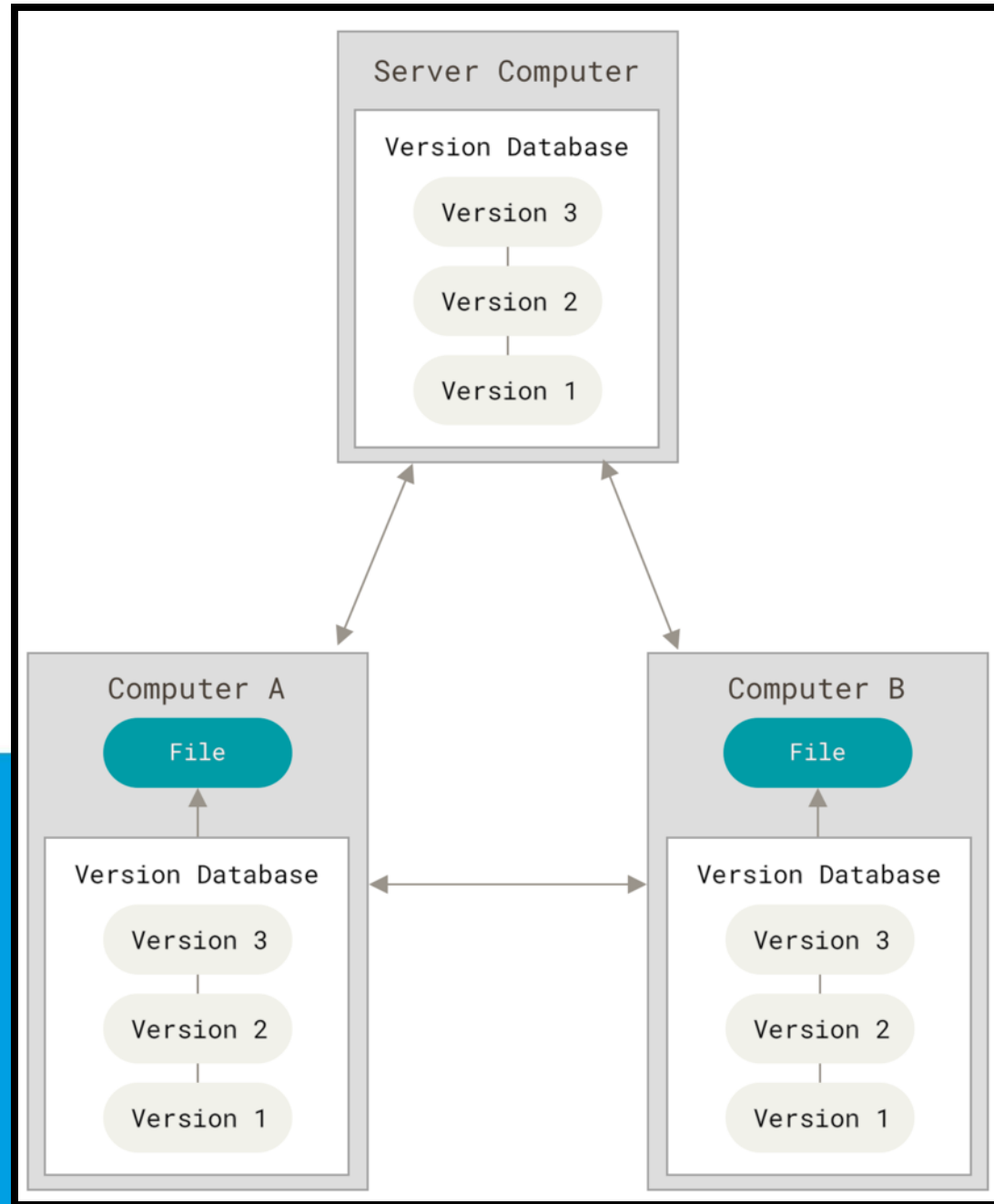
developer

developer

developer

VCS Distribuído

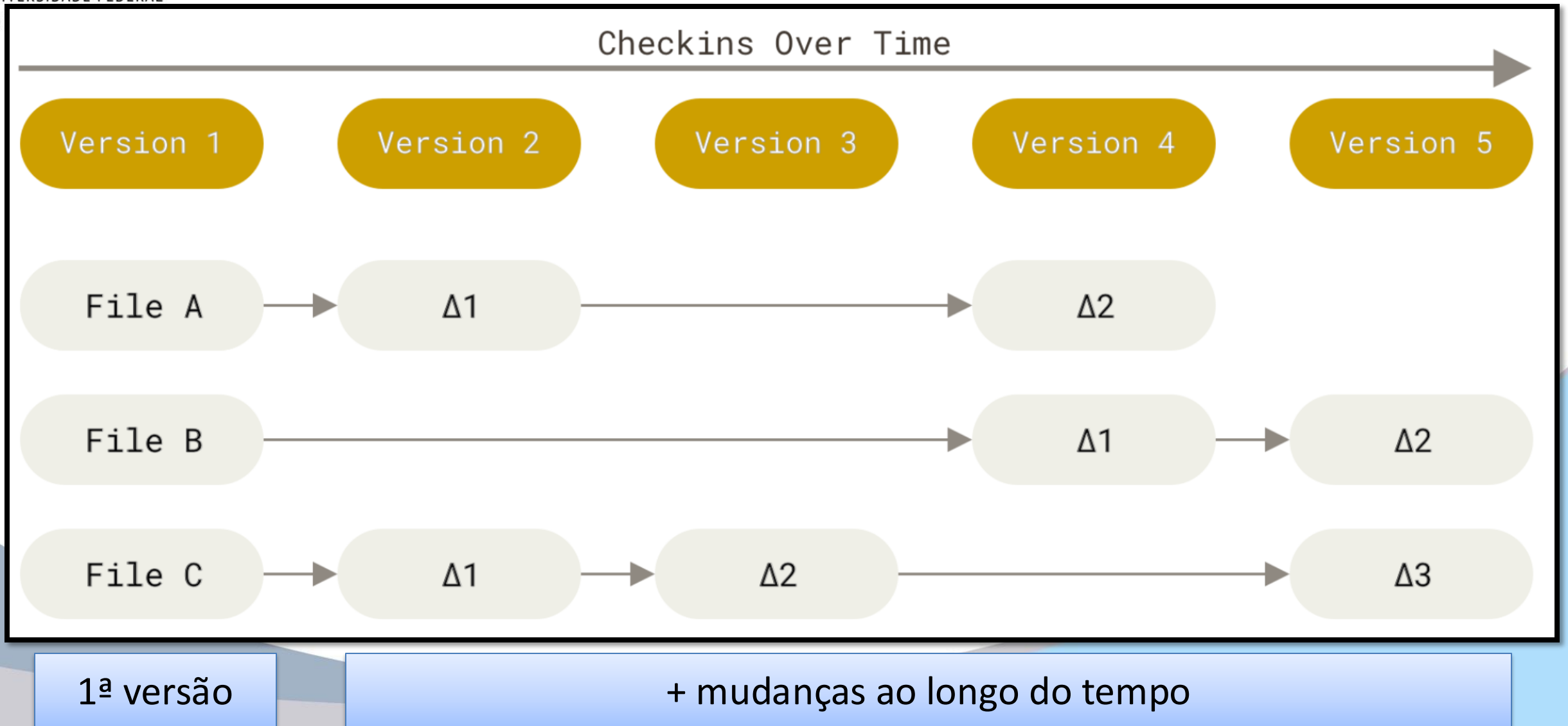
Ex.: Git, Mercurial, Bazaar, Darcs.



História do Git

- Projeto de desenvolvimento do Kernel do Linux:
 - *1991-2002: compartilhamento de patches;*
 - *2002: adoção do VCS distribuído BitKeeper;*
 - *2005: treta com a empresa do BitKeeper, desenvolvimento de ferramenta própria – o Git.*
- Características:
 - *Baseado nas lições aprendidas com o BitKeeper;*
 - *Velocidade;*
 - *Projeto simples;*
 - *Capaz de lidar com grandes projetos (ex.: Linux).*
 - *Forte suporte a ramificação;*
 - *Completamente distribuído;*

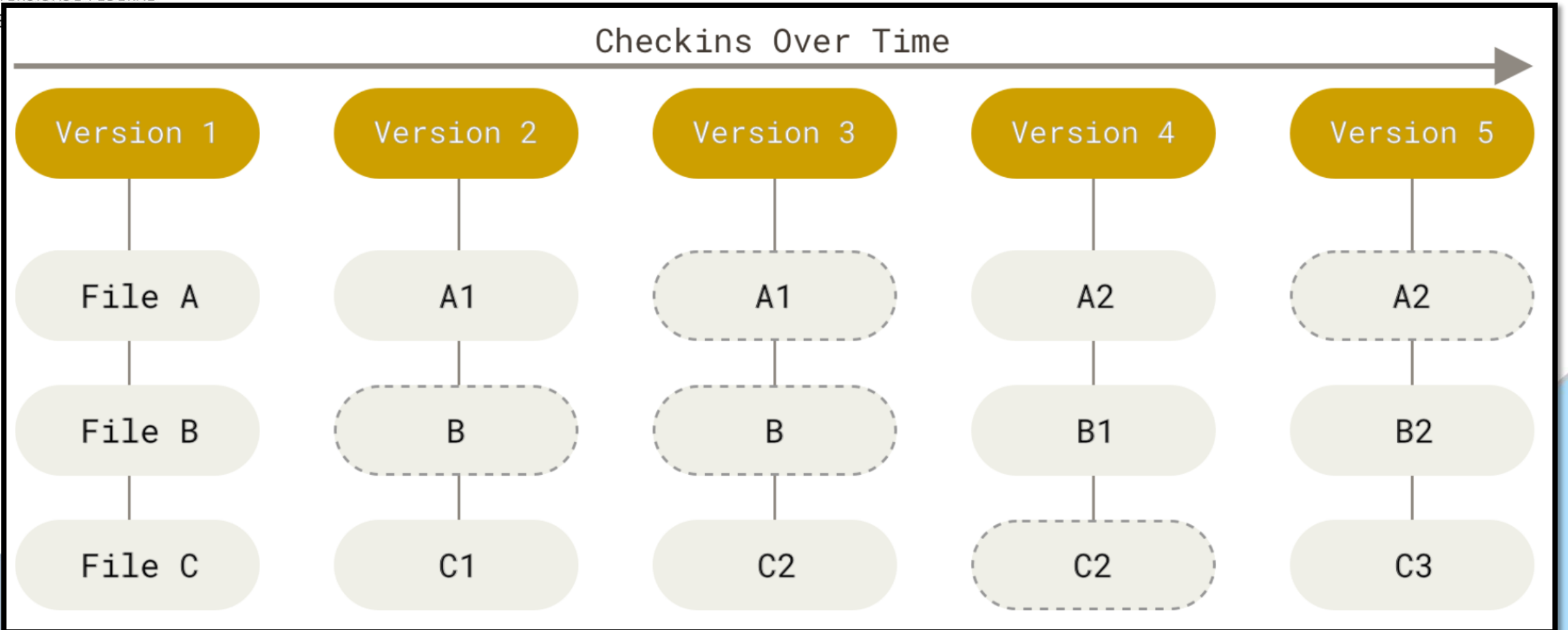
Como funcionavam os VCSs pré-Git



Como funciona o Git

É como um mini sistema de arquivos.

Checkins Over Time



1ª versão

A cada versão, “tira-se uma foto” dos arquivos que mudaram.

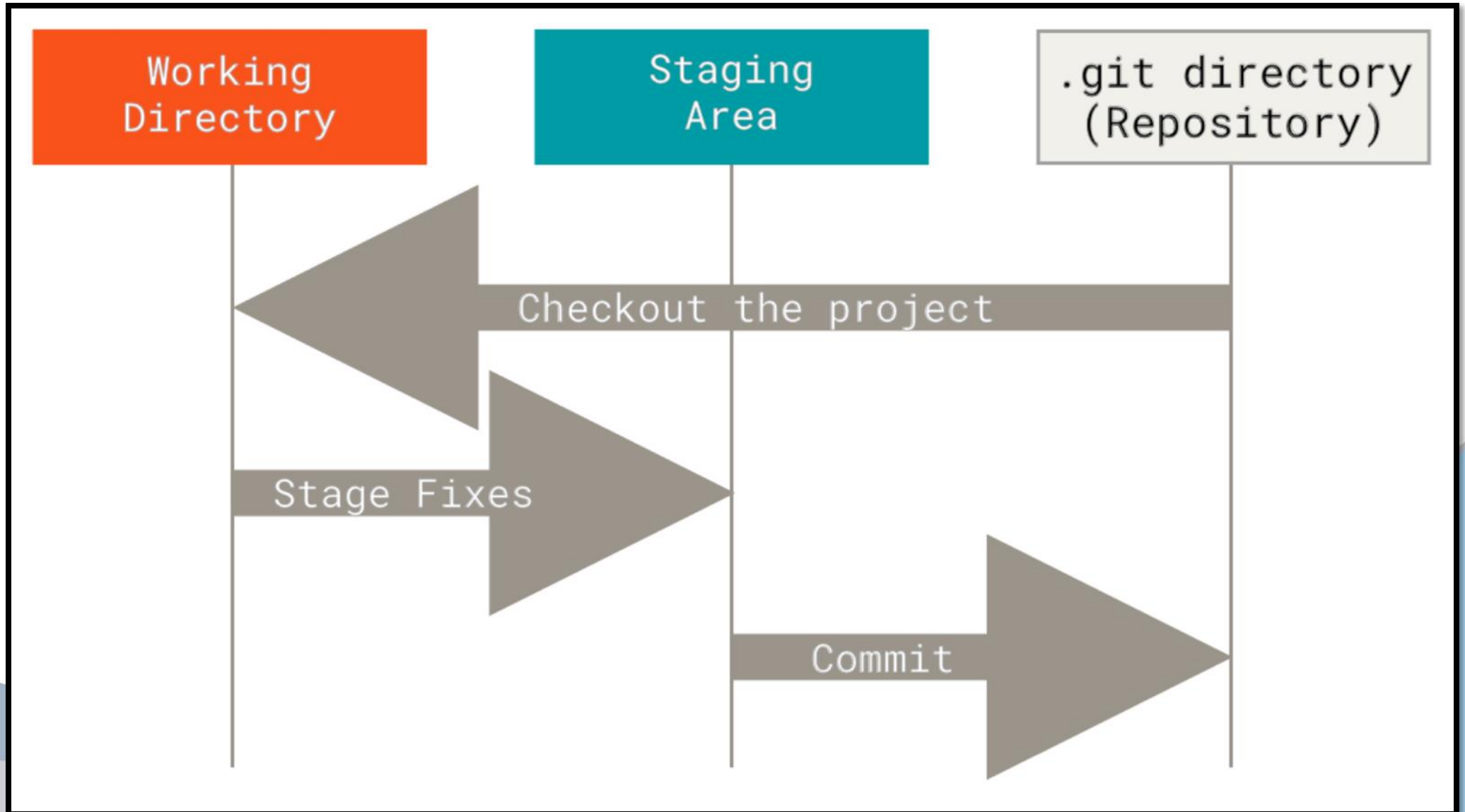
Como funciona o Git

- Quase todas as operações são locais:
 - *Evita a latência de rede;*
 - *Pode fazer muita coisa offline;*
- Git tem integridade:
 - *Tudo tem checksum (hash SHA-1);*
 - *Se algo for corrompido o Git detecta;*
- O git geralmente só adiciona dados:
 - *Difícil fazer algo que não seja reversível;*
 - *Muito difícil haver perdas.*

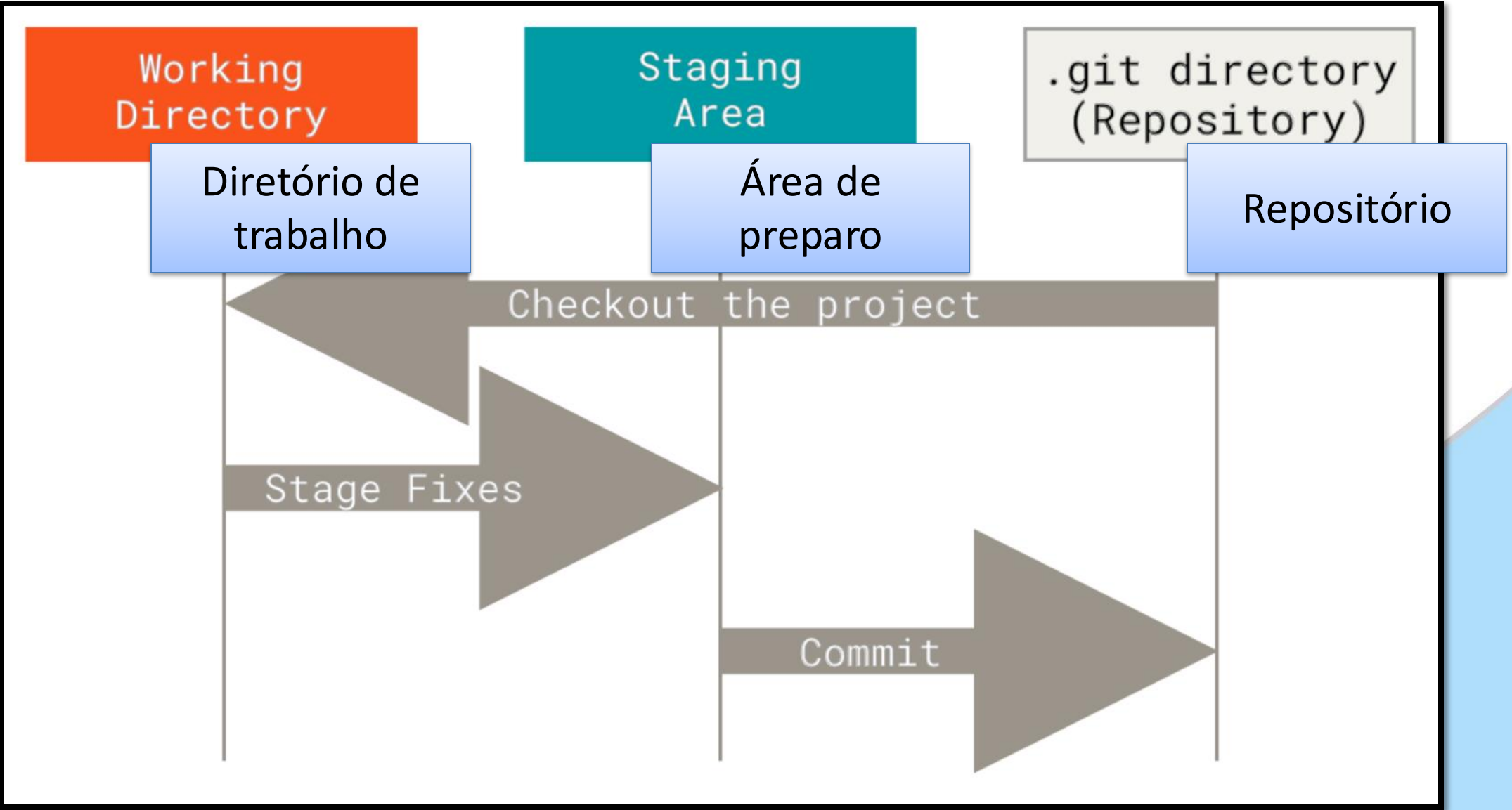
Os três estados do Git



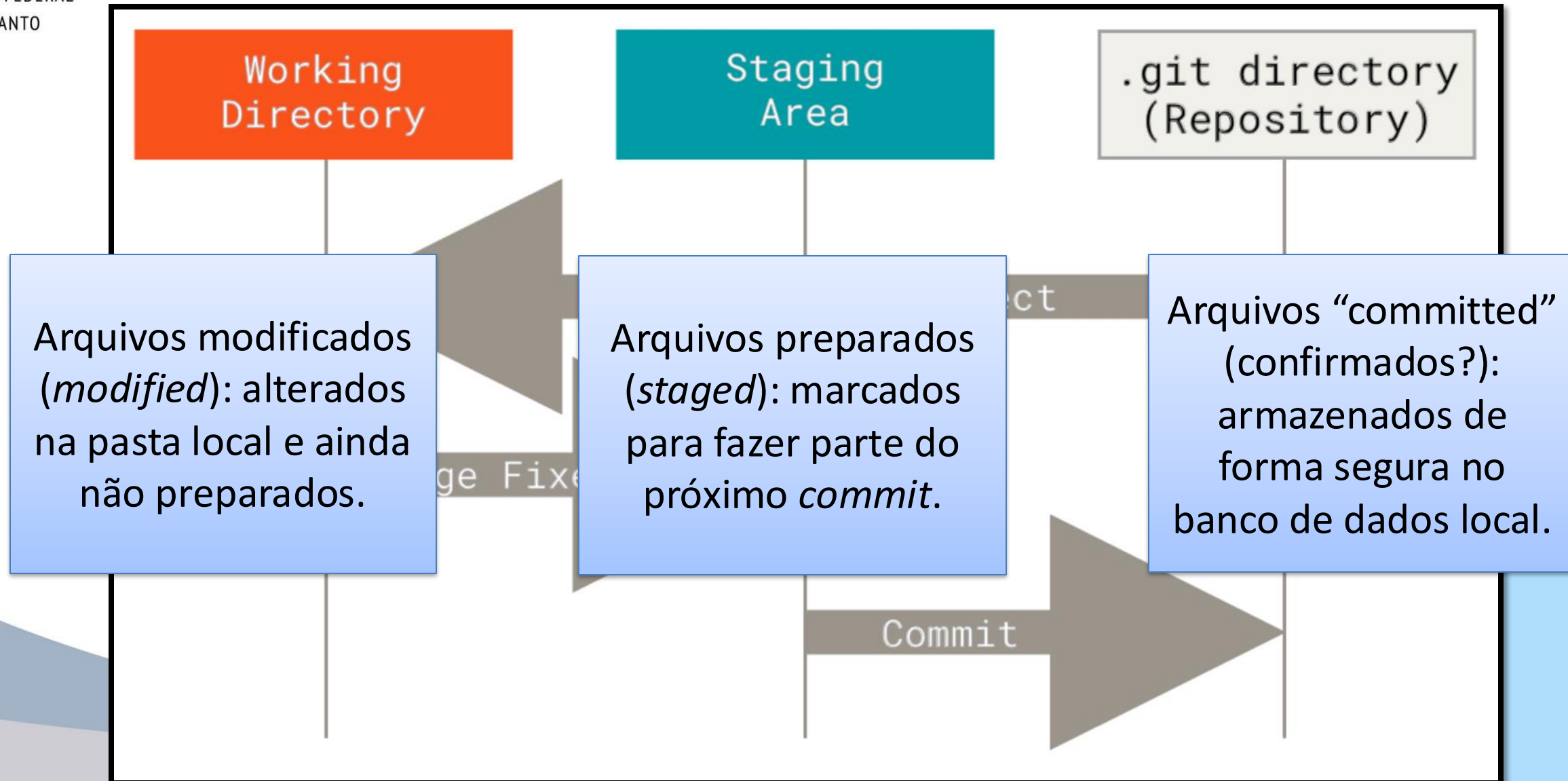
Muito
importante!



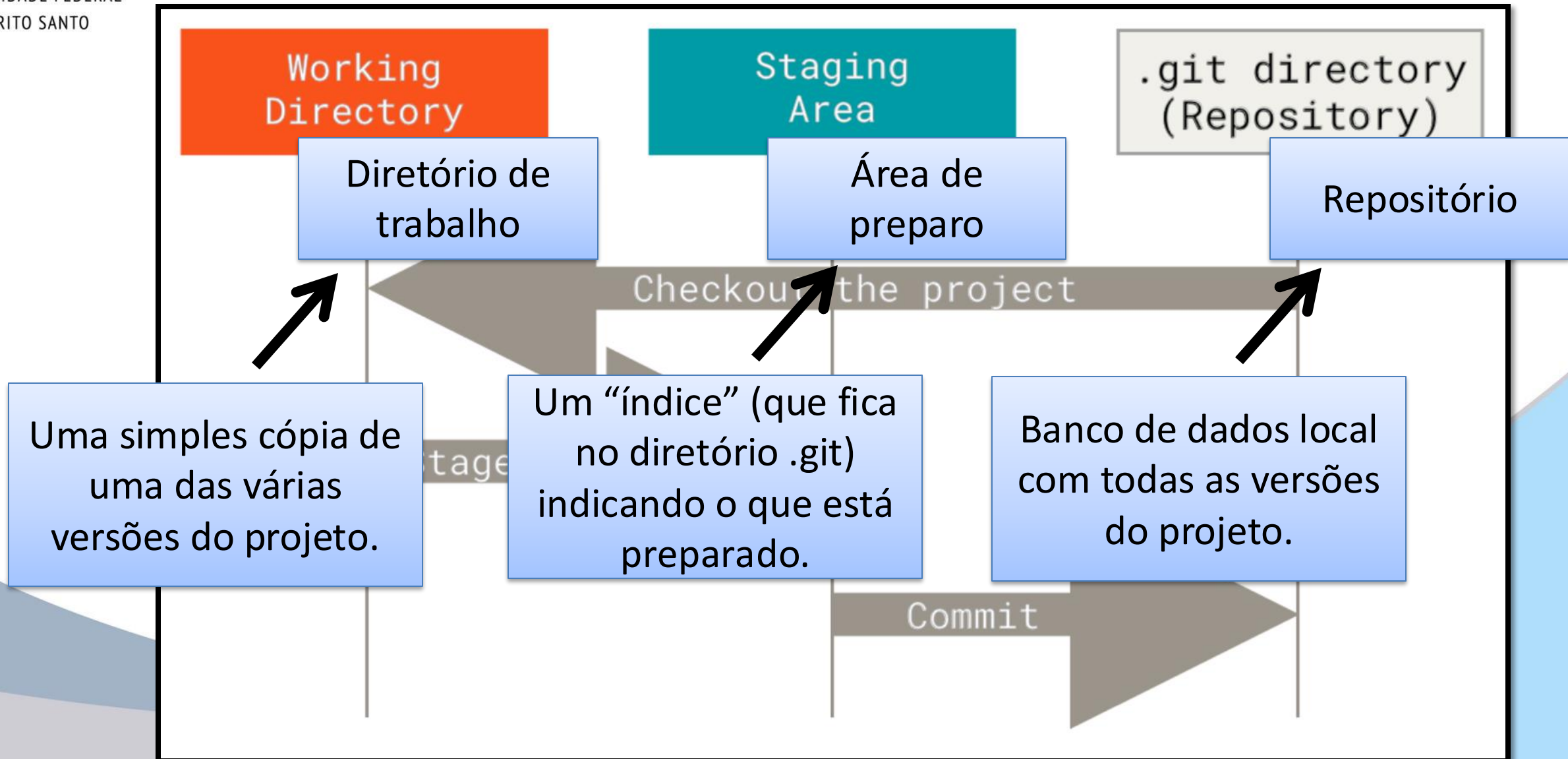
Os três estados do Git



Os três estados do Git



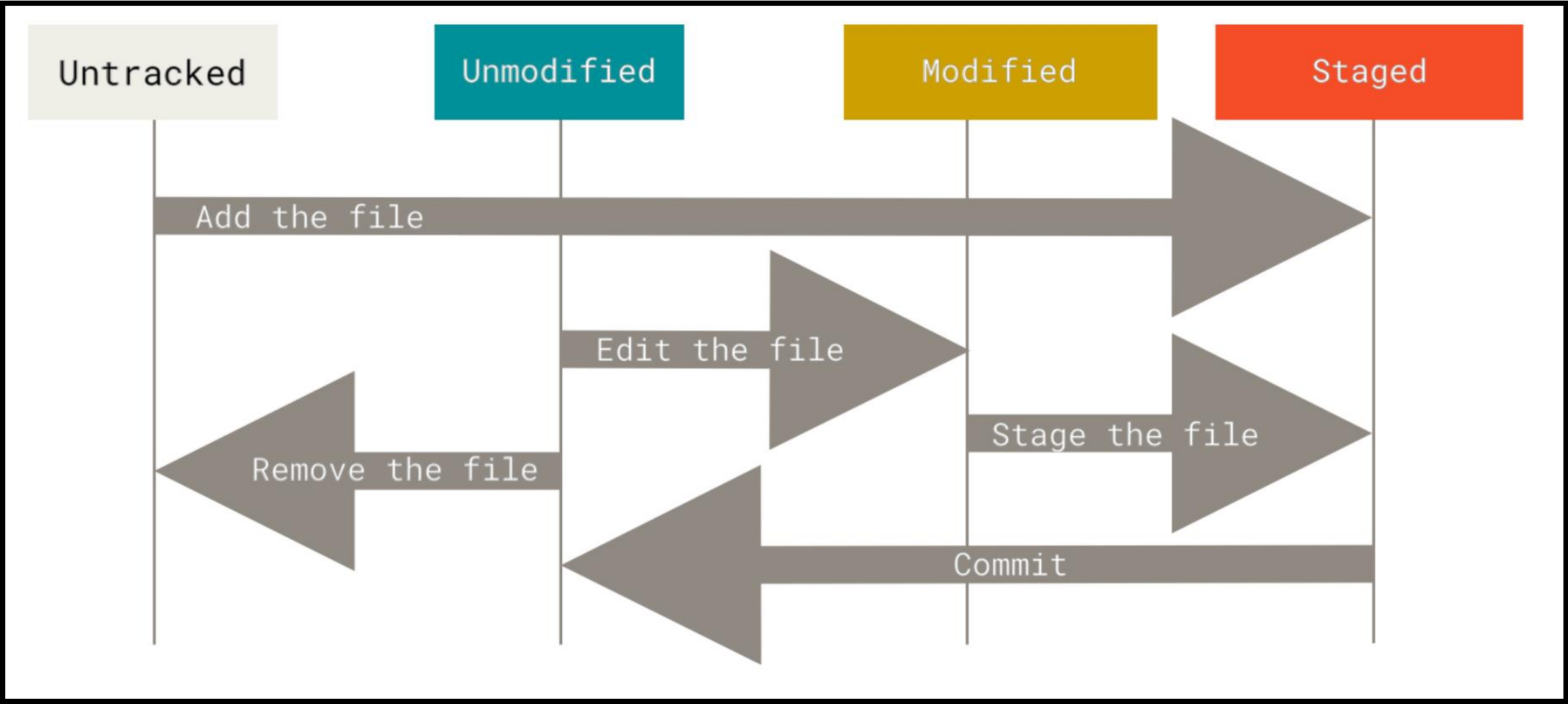
Os três estados do Git



Ciclo de vida básico

1. Você modifica arquivos no diretório de trabalho;
2. Você prepara os arquivos, adicionando suas imagens à área de preparo;
3. Você faz *commit*, o que leva as imagens preparadas para o banco de dados local (diretório .git).

Ciclo de vida básico



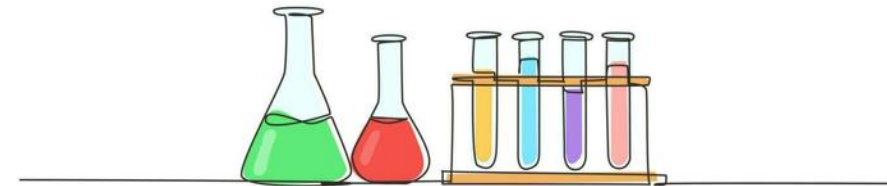
Instalação e configuração

- Versões Linux, MacOS e Windows:
 - <https://git-scm.com/downloads>
- Configuração básica:

```
git config --global user.email "fulano@exemplo.com.br"  
git config --global user.name "Fulano"  
git config --list
```

- *Opção --system grava para todos os usuários;*
- *Opção --global grava para todos os repositórios do usuário;*
- *Default grava apenas para o repositório atual.*

Roteiro de laboratório



- Faça a configuração básica do Git na sua máquina:

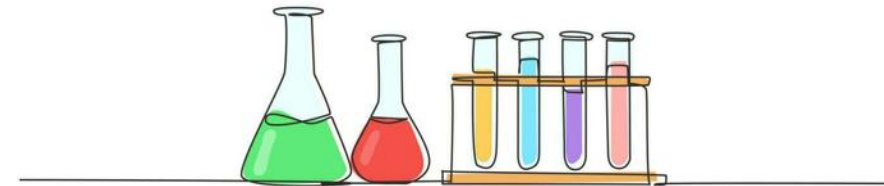
```
git config --global user.email "fulano@exemplo.com.br"
git config --global user.name "Fulano"
```

- Veja as configurações feitas:

```
git config user.email
git config user.name
git config --list
```

Sempre que vir essa figurinha,
experimente na sua máquina!

Criar um repositório



■ Novo repositório vazio:

```
vitor@localhost ~ % cd Workspaces/Live
```

```
vitor@localhost Live % mkdir git-test
```

```
vitor@localhost Live % cd git-test
```

```
vitor@localhost git-test % git init
```

```
Initialized empty Git repository in /Users/vitor/Workspaces/Live/git-test/.git/
```

- Pode ser criado numa pasta que já contenha arquivos;
- O diretório `.git` é criado;
- Porém nenhum arquivo ainda é monitorado.

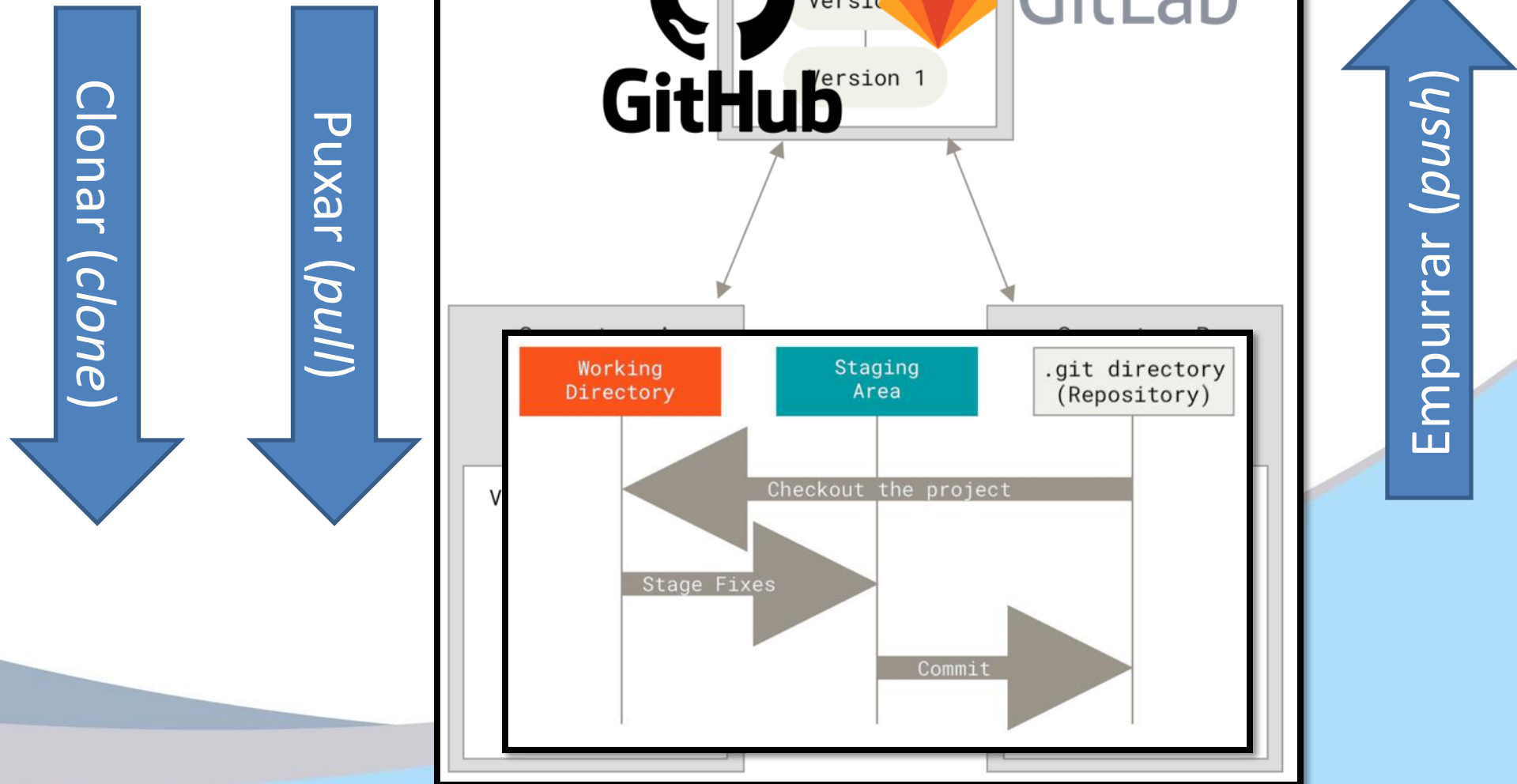
Obter um repositório remoto

- No jargão do Git, “clonar” um repositório:

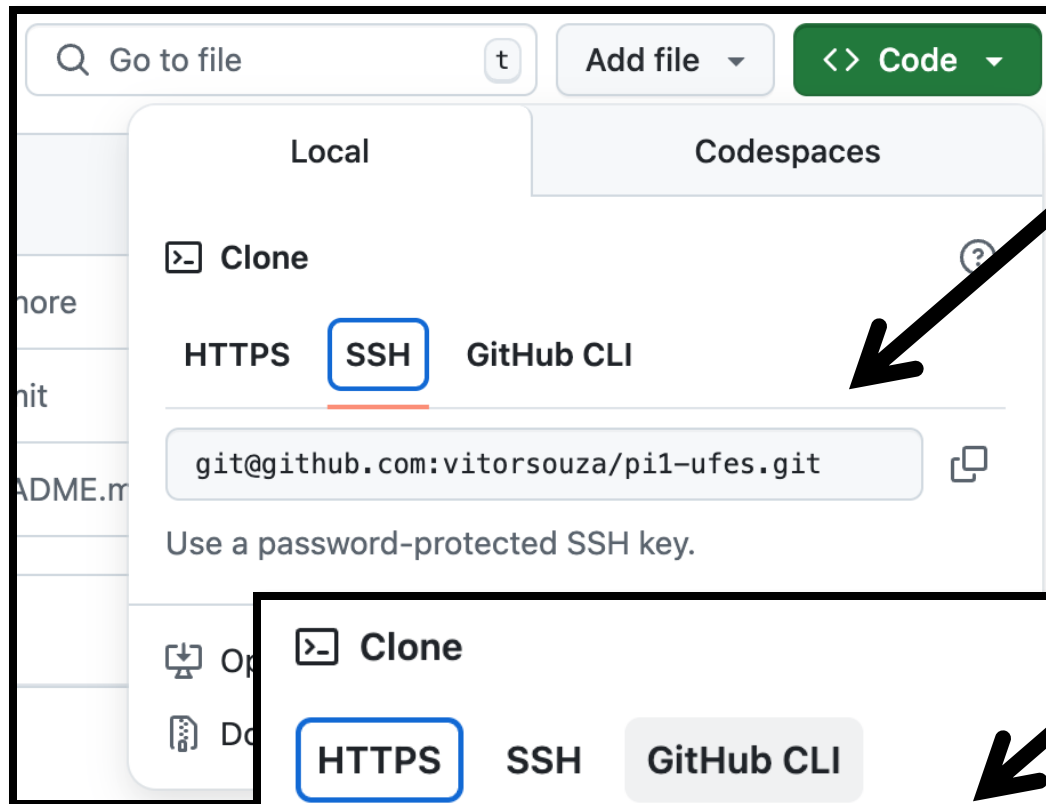
```
vitor@localhost ~ % cd Workspaces/Live
vitor@localhost Live % git clone git@github.com:vitorsouza/pi1-ufes.git
Cloning into 'pi1-ufes'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (4/4), 12.81 KiB | 257.00 KiB/s, done.
```

- Recebe-se a cópia completa do repositório e não apenas uma versão (*checkout*).

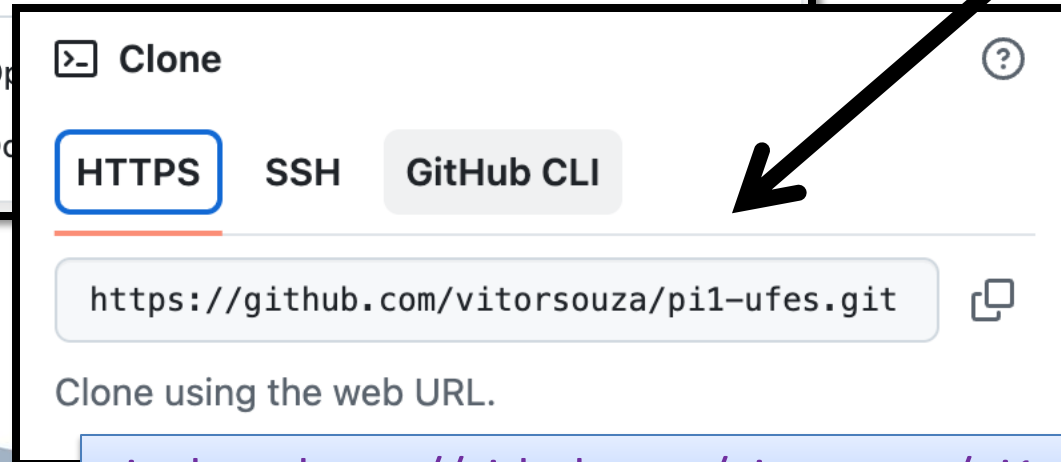
Repositórios remotos



Tipos de acesso remoto



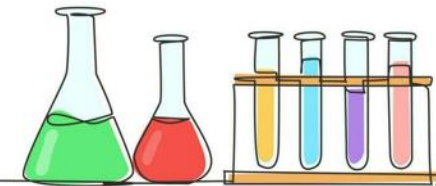
Requer autenticação via chaves assimétricas



Requer autenticação via login e senha

```
git clone https://github.com/vitorsouza/pi1-ufes.git
```


Obter um repositório remoto



- Tente clonar o repositório do professor via SSH:

```
git clone git@github.com:vitorsouza/pi1-ufes.git
```

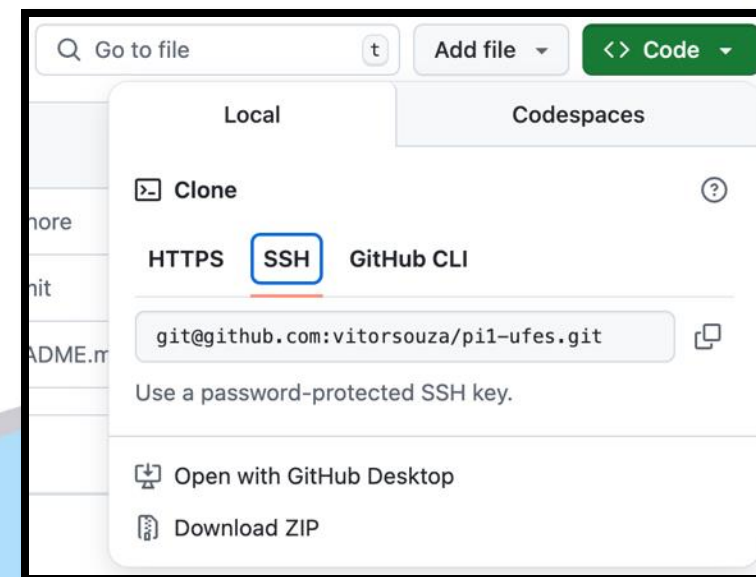
- O que acontece? Tente agora via HTTPS:

```
git clone https://github.com/vitorsouza/pi1-ufes.git
```

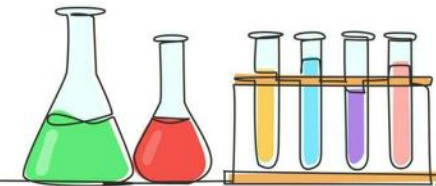
- O que acontece?

As URLs podem ser copiadas de

<https://github.com/vitorsouza/pi1-ufes>



Configurar chaves SSH

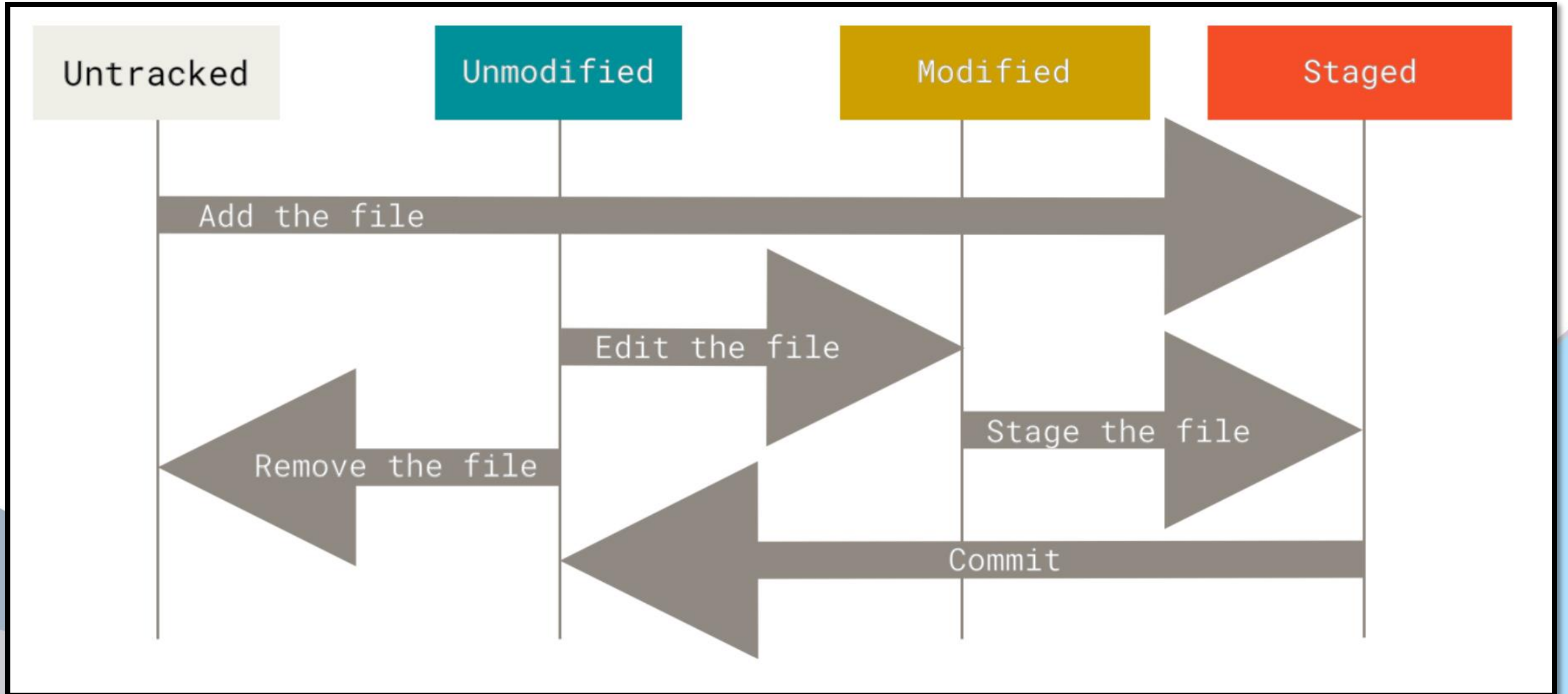


- Caso ainda não tenha, crie uma conta em github.com;
- Crie um par de chaves assimétricas (*public key, private key*), protegendo-a com senha:

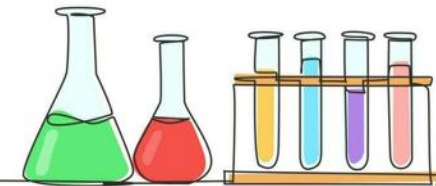
```
ssh-keygen -t ed25519 -C "fulano@exemplo.com.br"
```

- Copie o conteúdo da chave pública:
 - `~/.ssh/id_ed25519.pub`
- Adicione a chave ao GitHub:
 - Acesse <https://github.com/settings/keys> (ou clique no seu perfil > Settings > SSH and GPG keys);
 - Clique em New SSH Key e cole o conteúdo da chave pública.
- Crie um novo repositório com README e clone-o.

Relembrando o ciclo de vida básico



Status do repositório



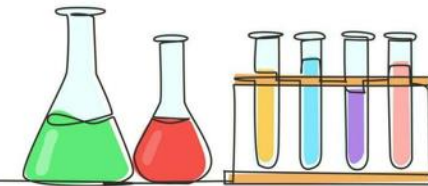
- Mostra que arquivos estão em que estado:

```
vitor@localhost pi1-ufes % git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

- Diretório limpo =
 - *Nenhum dos arquivos rastreado foi modificado; e*
 - *Não há arquivos não-rastreados.*
- Ramo (*branch*) main, antigamente master.

Status do repositório



- Se houver um arquivo não monitorado:

```
vitor@localhost pi1-ufes % vi Hello.java
```

```
vitor@localhost pi1-ufes % git status
```

On branch main

Your branch is up to date with 'origin/main'.

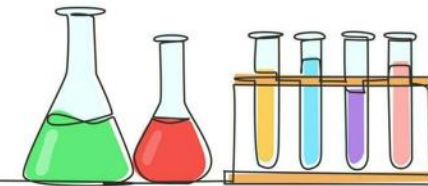
Untracked files:

(use "git add <file>..." to include in what will be committed)

Hello.java

nothing added to commit but untracked files present (use "git add" to track)

Adicionar (rastrear) um arquivo



- Adição de arquivos deve ser feita explicitamente:

```
vitor@localhost pi1-ufes % git add Hello.java
```

```
vitor@localhost pi1-ufes % git status
```

On branch main

Your branch is up to date with 'origin/main'.

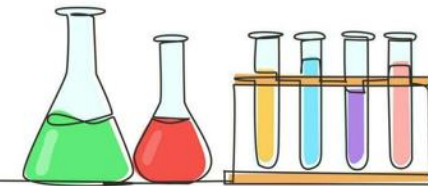
Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: Hello.java

- *O comando recebe arquivos e/ou diretórios (no segundo caso adiciona o conteúdo do diretório recursivamente);*
- *É feita uma foto dos arquivos neste momento. Se forem modificados, devem ser readicionados.*

Modificar um arquivo rastreado



- Resultado um pouco diferente de um novo arquivo:

```
vitor@localhost pi1-ufes % vi README.md
```

```
vitor@localhost pi1-ufes % git status
```

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: Hello.java

Changes not staged for commit:

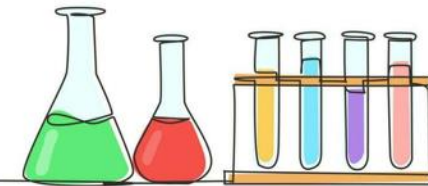
(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: README.md

Observe
algumas
instruções

Adicionar um arquivo rastreado



- Incorpore as modificações no arquivo já rastreado:

```
vitor@localhost pi1-ufes % git add README.md
```

```
vitor@localhost pi1-ufes % git status
```

On branch main

Your branch is up to date with 'origin/main'.

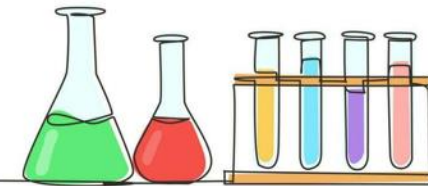
Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: Hello.java

modified: README.md

Adicionar e depois modificar



- Foto anterior foi adicionada, porém houve mudanças:

```
vitor@localhost pi1-ufes % vi README.md
```

```
vitor@localhost pi1-ufes % git status
```

On branch main

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

new file: Hello.java

modified: README.md

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

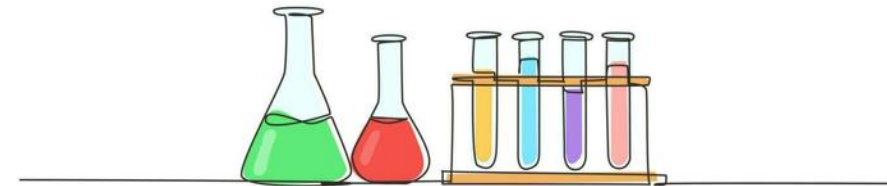
(use "git restore <file>..." to discard changes in working directory)

modified: README.md

Foto
anterior

Foto
nova

Status resumido



■ Opção -s ou --short:

```
vitor@localhost pi1-ufes % git status -s
```

```
A Hello.java
```

```
M README.md
```

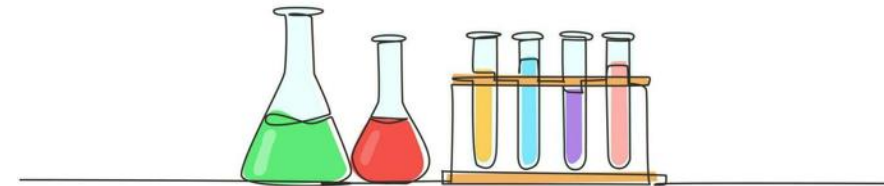
- *??: não rastreados;*
- *A: novos arquivos já preparados (staged);*
- *M: arquivos modificados na área de preparo (coluna da esquerda) e/ou no diretório de trabalho (direita).*

Múltiplos propósitos do git add

- Adicione um novo arquivo ainda não rastreado;
- Incorpore as modificações em arquivo já rastreado;
- Marque arquivo em conflito de mesclagem como resolvido...

Em resumo: “adicione este conteúdo ao próximo *commit*”.

Arquivos a ignorar



- Liste-os no arquivo `.gitignore`:

```
vitor@localhost pi1-ufes % javac Hello.java
```

```
vitor@localhost pi1-ufes % git status -s
```

```
A Hello.java
```

```
MM README.md
```

```
?? Hello.class
```

```
vitor@localhost pi1-ufes % echo *.class >> .gitignore
```

```
vitor@localhost pi1-ufes % git status -s
```

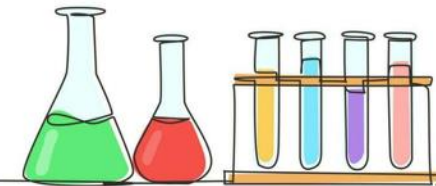
```
A Hello.java
```

```
MM README.md
```

```
?? .gitignore
```

- Veja exemplos: <https://github.com/github/gitignore>;
- É possível ter múltiplos arquivos `.gitignore`.

Ver o que foi modificado



- Veja o *diff* entre o diretório de trabalho e o *stage*:

```
vitor@localhost pi1-ufes % git diff
```

```
diff --git a/README.md b/README.md
```

```
index fd9c1b3..252a619 100644
```

```
--- a/README.md
```

```
+++ b/README.md
```

```
@@ -1,2 +1,2 @@
```

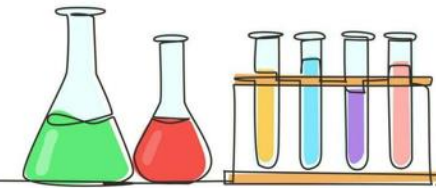
```
-# Projeto Integrado I / 2024-1
```

```
+# Projeto Integrado I - 2024/1
```

```
Repositório para testes da disciplina de Projeto Integrado I, semestre 2024/1 na UFES
```

- *Se tudo já estiver preparado, a saída será vazia!*
- *Experimente também: `git difftool`.*

Ver o que foi preparado



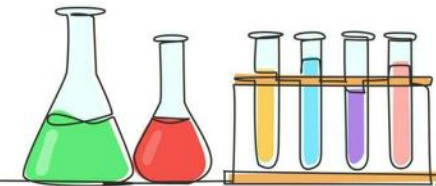
- Veja o *diff* entre o *stage* e o último *commit*:

```
vitor@localhost pi1-ufes % git diff --staged
diff --git a/Hello.java b/Hello.java
new file mode 100644
index 0000000..95a9e05
--- /dev/null
+++ b/Hello.java
@@ -0,0 +1,6 @@
+public class Hello {
+    public static void main(String[] args) {
+        System.out.println("Hello, World!");
+    }
+}
```


Enfim, *commit*!

- Estando satisfeito com a área de preparo (*stage*):
 - *git commit*: abre um editor para inclusão de mensagem;
 - *git commit -v*: inclui o diff na mensagem no editor;
 - *git commit -m "mensagem"*: já passa a mensagem;
 - *git commit -a*: adiciona todos os arquivos rastreados à área de preparo automaticamente.
- O retorno do comando indica um resumo do que aconteceu e inclui um *hash* SHA-1 do *snapshot*.

Fazer *commit*



- Opção `-a` não inclui arquivos não rastreados:

```
vitor@localhost pi1-ufes % git commit -a -m "Algumas mudanças"
```

```
[main 5517765] Algumas mudanças
```

```
2 files changed, 7 insertions(+), 1 deletion(-)
```

```
create mode 100644 Hello.java
```

```
vitor@localhost pi1-ufes % git status -s
```

```
?? .gitignore
```

```
vitor@localhost pi1-ufes % git add .gitignore
```

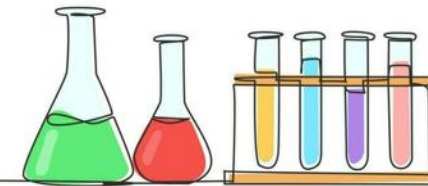
```
vitor@localhost pi1-ufes % git commit -m "Inclui .gitignore"
```

```
[main 79255eb] Inclui .gitignore
```

```
1 file changed, 1 insertion(+)
```

```
create mode 100644 .gitignore
```

Remover arquivos



- Remover do dir. trabalho vs. remover do *stage*:

```
vitor@localhost pi1-ufes % rm Hello.*
```

```
vitor@localhost pi1-ufes % git status -s
```

```
D Hello.java
```

```
vitor@localhost pi1-ufes % git rm Hello.java
```

```
rm 'Hello.java'
```

```
vitor@localhost pi1-ufes % git status -s
```

```
D Hello.java
```

```
vitor@localhost pi1-ufes % git commit -m "Remove Hello"
```

```
[main c9a7c94] Remove Hello
```

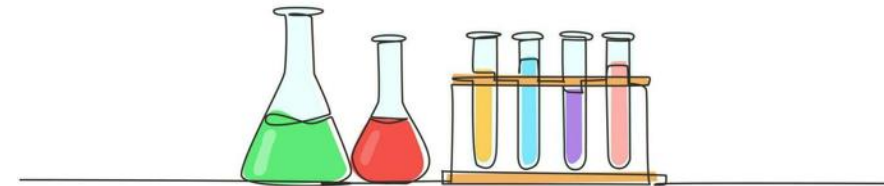
```
1 file changed, 6 deletions(-)
```

```
delete mode 100644 Hello.java
```

Mover e renomear arquivos

- O Git não rastreia explicitamente a movimentação de arquivos;
 - *Porém costuma ser bastante esperto para perceber que algo foi renomeado ou movido;*
- Comando `git mv origem destino`, equivalente a:
 - *`mv origem destino`*
 - *`git rm origem`*
 - *`git add destino`*

Histórico de *commits*



- Listar os últimos *commits*:
 - *git log*: mostra hash, autor, data e texto dos *commits*;
 - *git log -p*: inclui os *diffs* de cada *commit*;
 - *git log -2*: limita aos 2 últimos *commits*;
 - *git log --stat*: inclui estatísticas de cada *commit*;
 - *git log --graph*: mostra grafo de *commits*;
 - *git log --pretty=oneline*: mostra *commits* em uma linha. Experimente *short*, *full* e *fuller*;
 - *git log --pretty=format:"%h - %an, %ar : %s"*: formato personalizado.

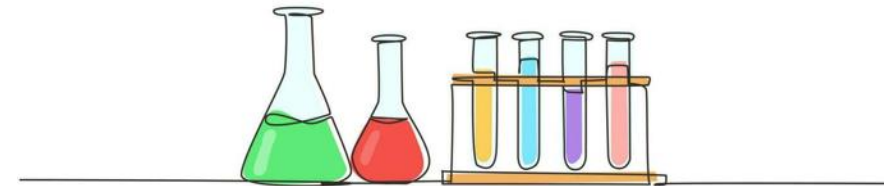
Muitas outras opções!

Desfazendo coisas

- `git commit --amend`: adiciona os novos conteúdos ao commit anterior (emenda);
- `git reset HEAD <arquivo>`: remove arquivo do *stage*;
- `git checkout -- <arquivo>`: descarta as modificações no arquivo (substitui pela versão no repositório).

Veja também as dicas do próprio git no slide 33!

Trabalhar com remotos



■ Sincronizando com o GitHub:

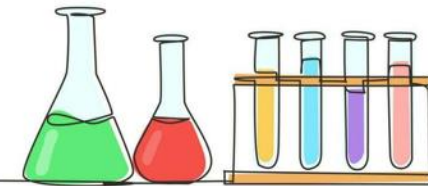
```
vitor@localhost pi1-ufes % git remote -v
origin git@github.com:vitorsouza/pi1-ufes.git (fetch)
origin git@github.com:vitorsouza/pi1-ufes.git (push)

vitor@localhost pi1-ufes % git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 949 bytes | 474.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0)
remote: Resolving deltas: 100% (2/2), done.
To github.com:vitorsouza/pi1-ufes.git
db2cda3..c9a7c94 main -> main
```


Repositórios remotos

- Nome padrão “origin” atribuído na clonagem;
- `git remote add <nome> <url>`: adiciona remoto;
- `git fetch [nome]`: extrai do repositório remoto todos os dados que você ainda não tenha;
- `git pull [nome]`: além de extrair os dados, mescla automaticamente o ramo remoto com o local (se ele estiver configurado para rastreá-lo);
- `git push [nome] [ramo]`: envia *commits*;
 - *Remoto “origin” e ramo atual são usados como padrão caso [nome] e/ou [ramo] não forem especificados.*

Trabalhar com remotos



- Modifique o README no GitHub e depois sincronize:

```
vitor@localhost pi1-ufes % git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), done.
From github.com:vitorsouza/pi1-ufes
   c9a7c94..fa465ef  main    -> origin/main
Updating c9a7c94..fa465ef
Fast-forward
 README.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Repositórios remotos (mais)

- `git remote show <nome>`: mostra informações;
- `git remote rename [nome] [novo-nome]`: renomeia um repositório remoto;
- `git remote remove [nome]`: remove a ligação com o repositório remoto no repositório local.

Tags

- Podem ser usadas para marcar pontos importantes no histórico (ex.: um *release*):
 - *git tag*: lista as tags;
 - *git tag -l "v1.*"*: filtra as tags;
 - *git tag -a <tag> -m "<mensagem>"*: adiciona tag;
 - *git tag <tag>*: adiciona tag leve (sem mensagem, checksum, usuário e data);
 - *git tag -a <tag> <commit>*: adiciona a tag um commit passado;

Tags

- Continua:

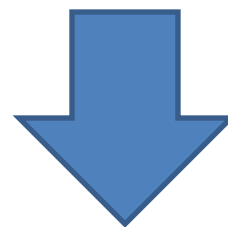
- *git push origin <tag>*: envia a tag ao repositório remoto “origin” (apenas *git push* não envia tags);
- *git push origin --tags*: envia todas as tags;
- *git checkout -b <branch> <tag>*: cria um ramo a partir do commit que possui a tag especificada.

(Veremos mais sobre ramos depois)

Apelidos (aliases)

- Você pode adicionar atalhos para os comandos que mais utiliza:

```
git config --global alias.st status
git config --global alias.unstage 'reset HEAD --'
git config --global alias.last 'log -1 HEAD'
```



```
git st
git unstage arquivo
git last
```

=

```
git status
git reset HEAD -- arquivo
git log -1 HEAD
```

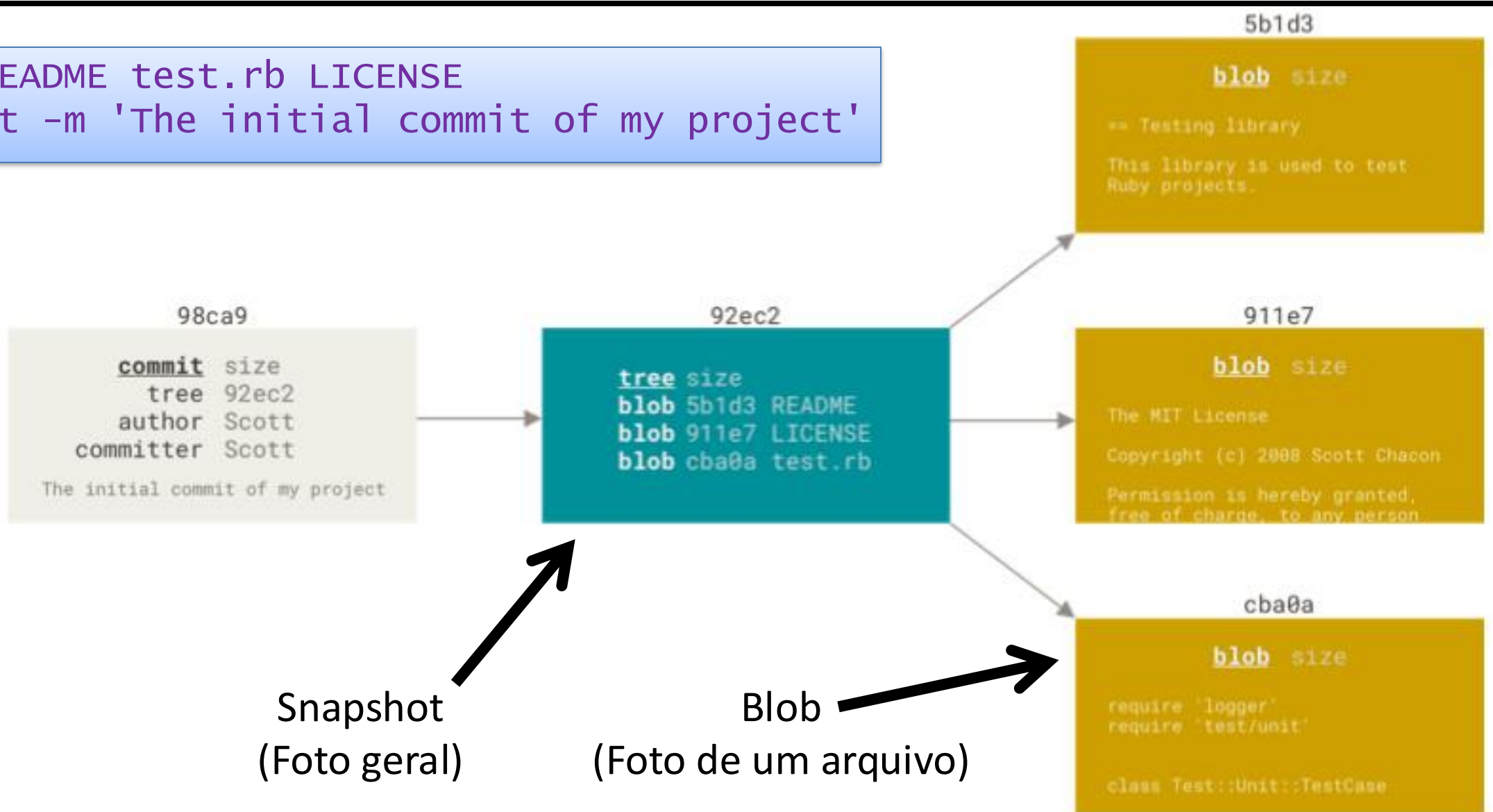
Ramificação (*branching*) no Git

Ramificação (*branching*)

- Divergir da linha principal de desenvolvimento:
 - *Trabalhar em algo específico sem alterar a main;*
 - *Após concluído, mesclar ou descartar o ramo.*
- Diferencial do Git: ramificação leve, muito eficiente:
 - *Commits são ponteiros para snapshots + metadados;*
 - *Commits apontam para seus pais;*
 - *Commits podem ramificar (2 commits com o mesmo pai);*
 - *Um ramo é um ponteiro móvel para um dos commits.*

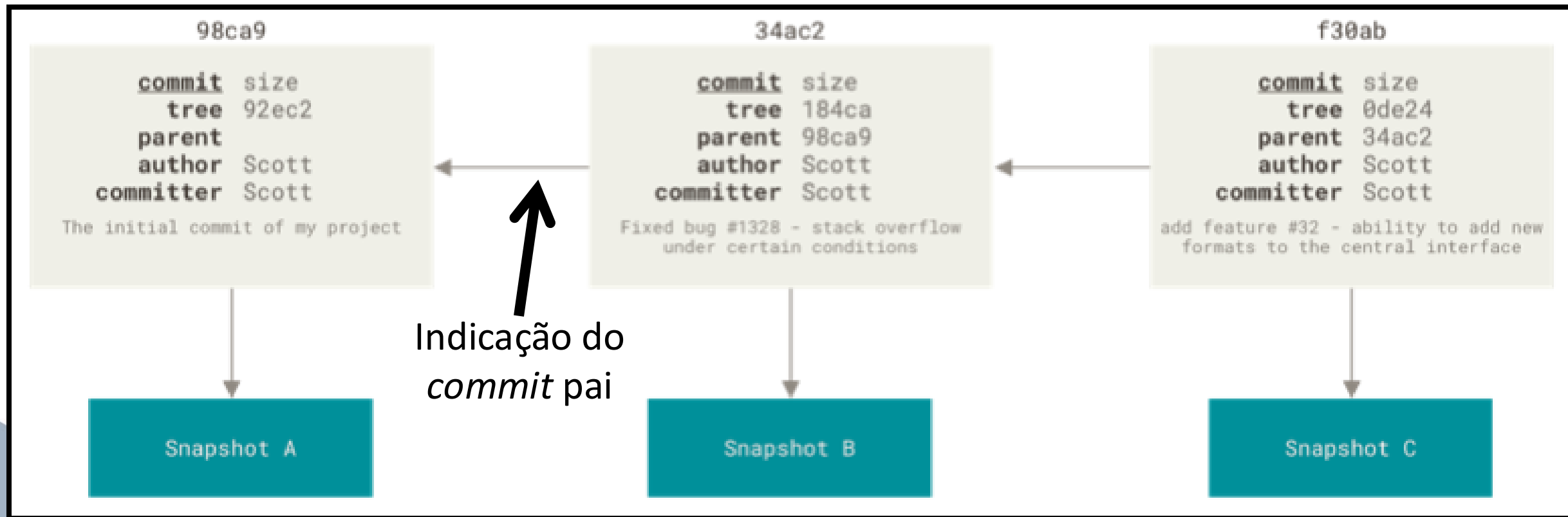
Por baixo dos panos

```
git add README test.rb LICENSE
git commit -m 'The initial commit of my project'
```

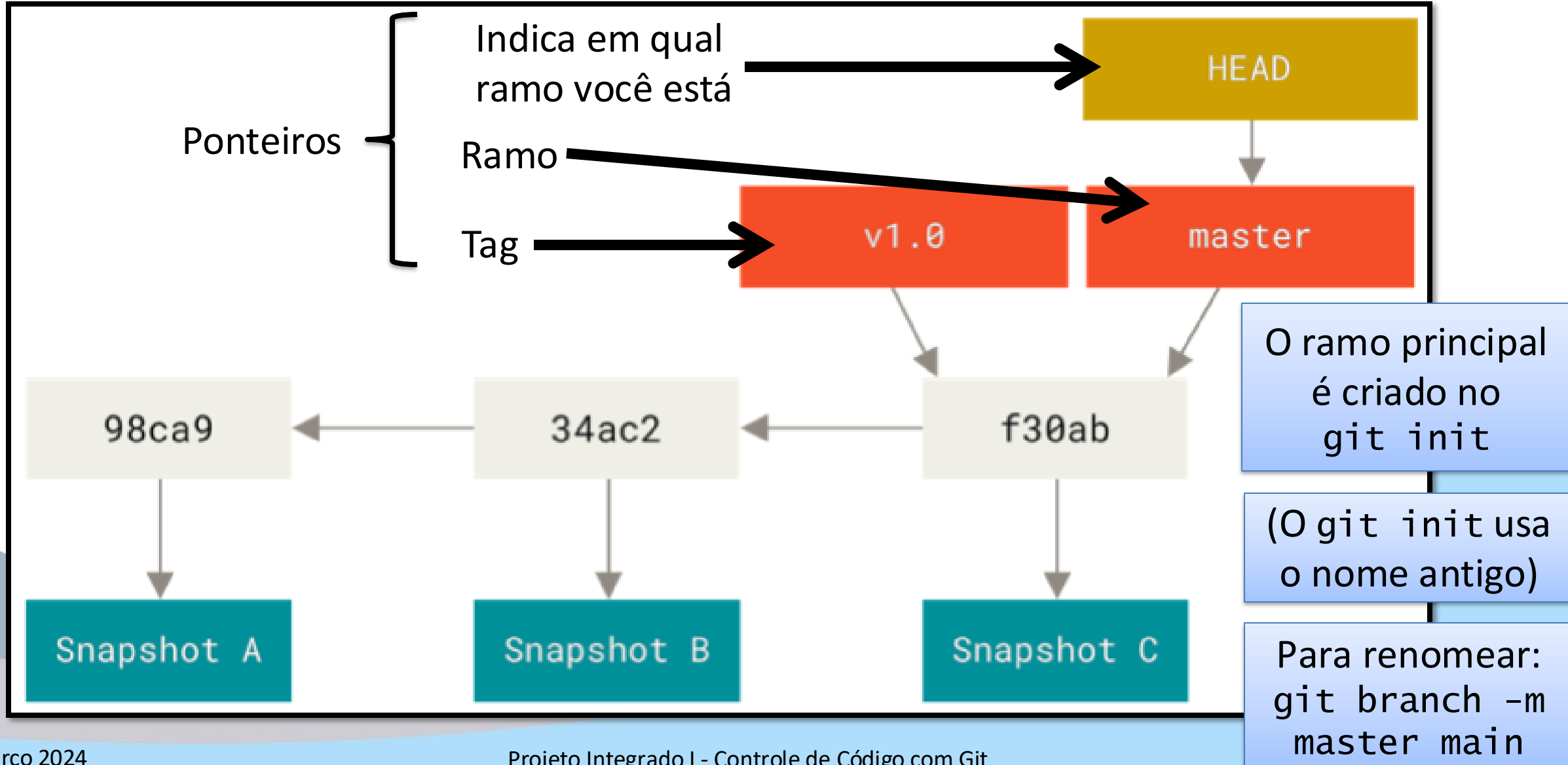


Por baixo dos panos

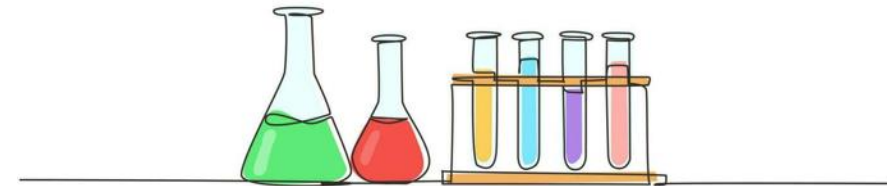
- Dois *commits* depois...



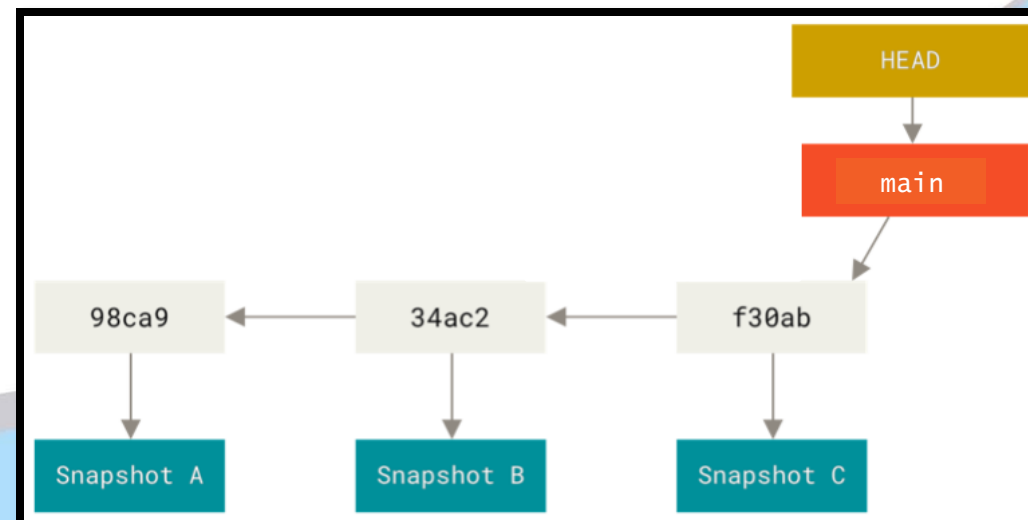
Por baixo dos panos



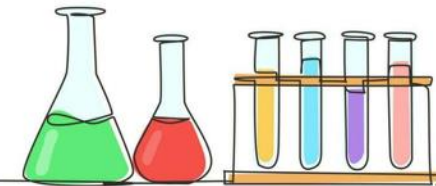
Preparando o experimento



- Experimente na sua máquina:
 1. *Crie um repositório vazio;*
 2. *Crie um arquivo de licença, um arquivo de código-fonte (Hello, World!) e um README para seu projeto;*
 3. *Faça o commit inicial com os 3 arquivos;*
 4. *Faça mais um ou dois commits com alterações em algum destes arquivos.*



Criar novo ramo

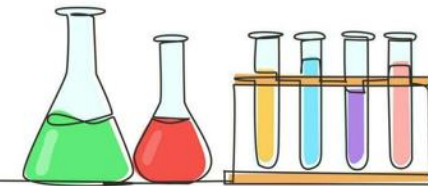


- Crie um novo ramo chamado testing:

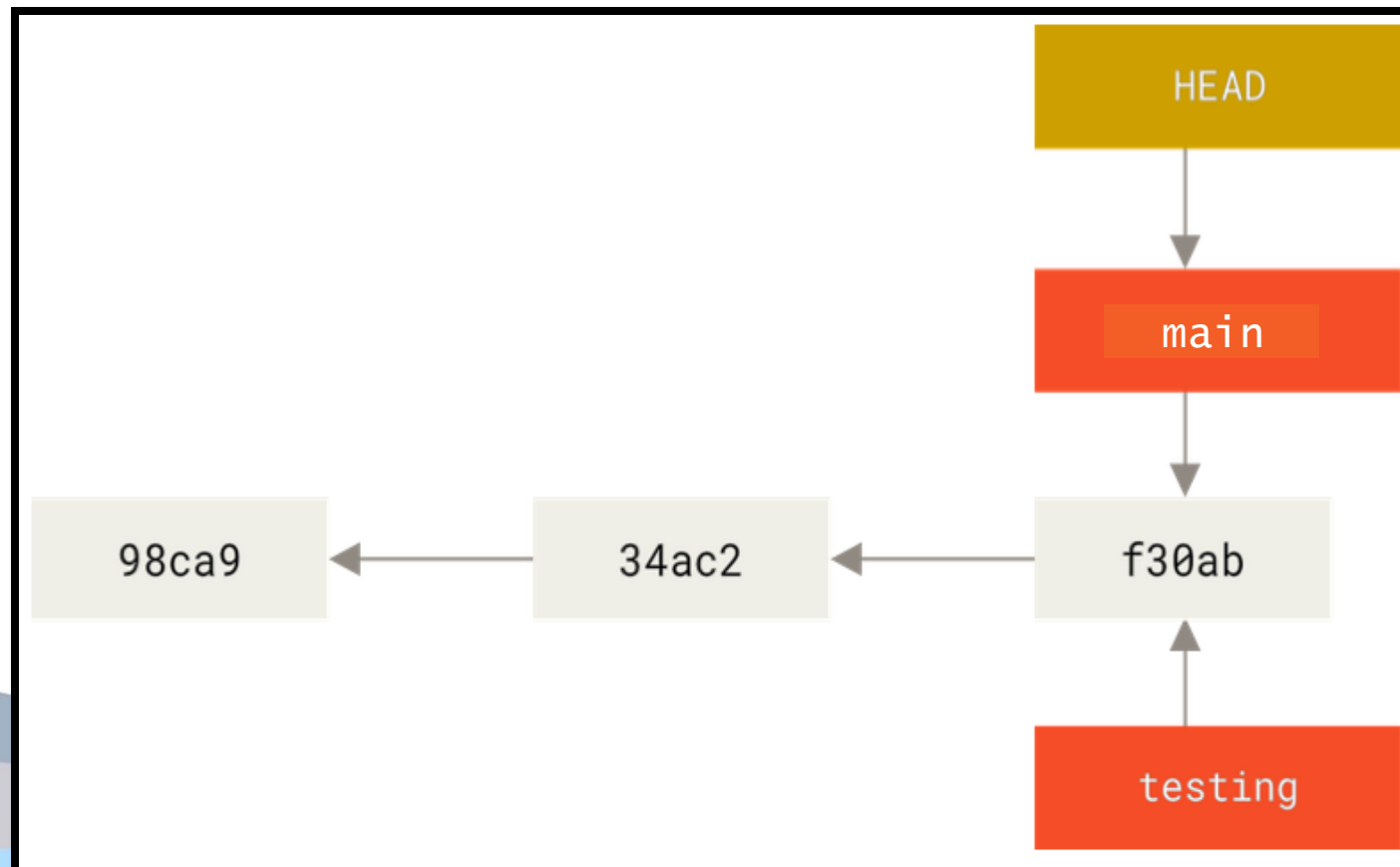
```
vitor@localhost repo % git branch testing
```

Pergunta: onde está o
ponteiro HEAD?

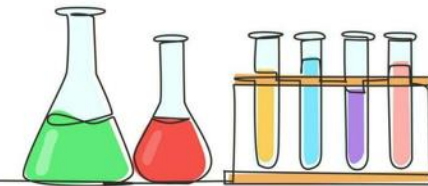
Verificar onde está o HEAD



```
vitor@localhost repo % git log --oneline --decorate
1d2db7b (HEAD -> main, testing) Personaliza Hello, World!
a715148 Adiciona comentário à classe
1f3ee0e Commit inicial
```



Alternar entre ramos

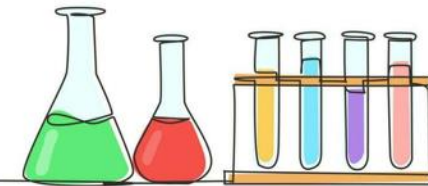


```
vitor@localhost repo % git checkout testing
Switched to branch 'testing'
```

```
vitor@localhost repo % git log --oneline --decorate
1d2db7b (HEAD -> testing, main) Personaliza Hello, World!
a715148 Adiciona comentário à classe
1f3ee0e Commit inicial
```



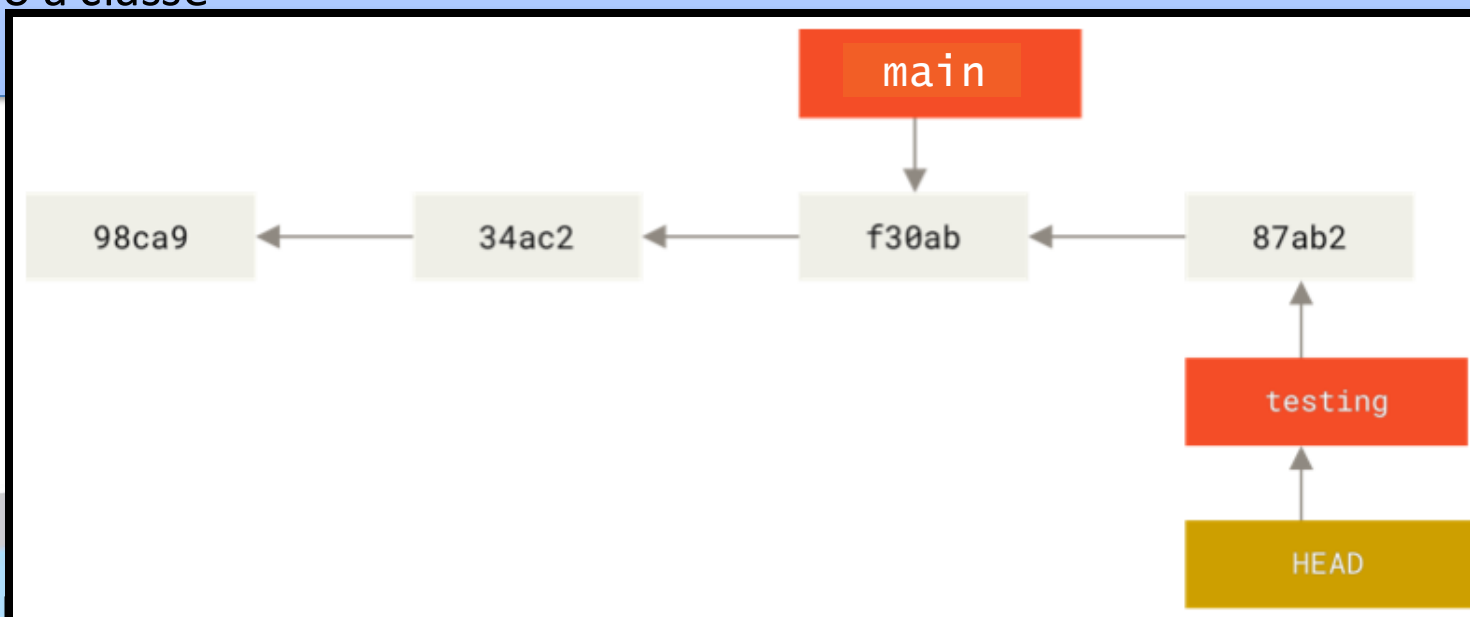
Avançar em um ramo



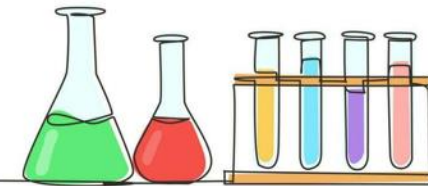
```
vitor@localhost repo % vi HelloWorld.java
vitor@localhost repo % git commit -a -m "Traduz para Olá, Mundo"
[testing 4d9e107] Traduz para Olá, Mundo
1 file changed, 2 insertions(+), 2 deletions(-)
```

```
vitor@localhost repo % git log --oneline --decorate

4d9e107 (HEAD -> testing) Traduz para Olá, Mundo
1d2db7b (main) Personaliza Hello, World!
a715148 Adiciona comentário à classe
1f3ee0e Commit inicial
```



Seguir numa direção diferente



```
vitor@localhost repo % git checkout main
```

```
Switched to branch 'main'
```

```
vitor@localhost repo % vi HelloWorld.java
```

```
vitor@localhost repo % git commit -a -m "Comenta método main"
```

```
[main 3b0f42a] Comenta método main
```

```
1 file changed, 1 insertion(+)
```

```
vitor@localhost repo % git log --oneline --decorate --graph --all
```

```
* 3b0f42a (HEAD -> main) Comenta método main
```

```
| * 4d9e107 (testing) Traduz para Olá, Mundo
```

```
|/
```

```
* 1d2db7b Personaliza Hello, World!
```

```
* a715148 Adiciona comentário à classe
```

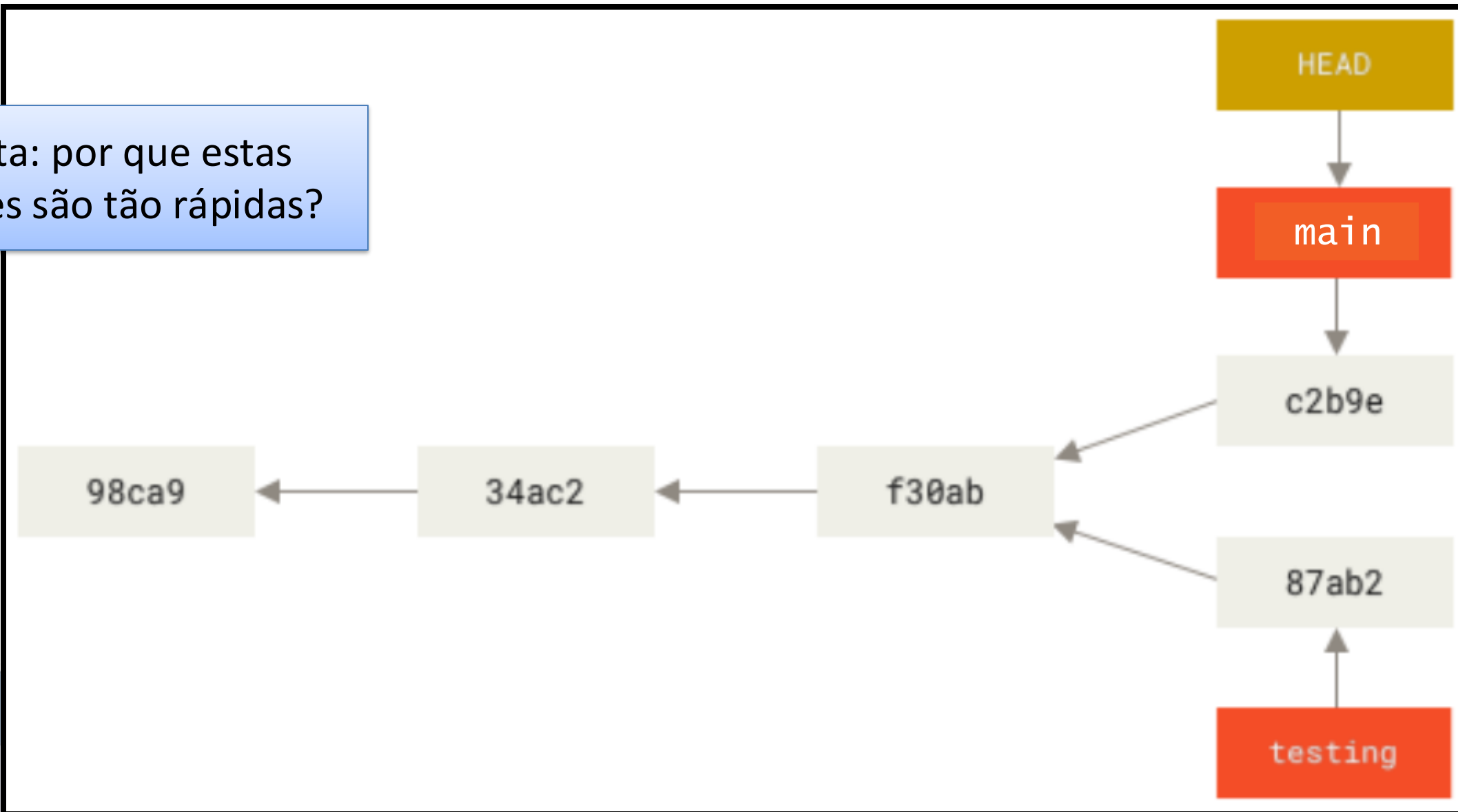
```
* 1f3ee0e Commit inicial
```

Confira o
conteúdo do
arquivo
modificado
em testing!

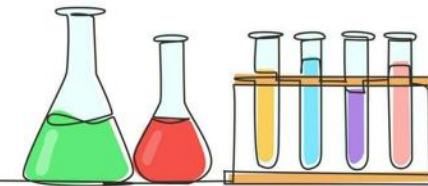
Voltar da testing para a main: apenas
com diretório de trabalho limpo.

Ramificações

Pergunta: por que estas operações são tão rápidas?



Evite *commits* na main



Atalho para
git branch
+
git
checkout

```
vitor@localhost repo % git checkout -b hotfix
```

```
Switched to a new branch 'hotfix'
```

```
vitor@localhost repo % vi HelloWorld.java
```

```
vitor@localhost repo % git commit -a -m "Corrige comentário"
```

```
[hotfix be33286] Corrige comentário
```

```
1 file changed, 4 insertions(+), 1 deletion(-)
```

```
vitor@localhost repo % git log --oneline --decorate --graph --all
```

```
* be33286 (HEAD -> hotfix) Corrige comentário
```

```
* 3b0f42a (main) Comenta método main
```

```
| * 4d9e107 (testing) Traduz para Olá, Mundo
```

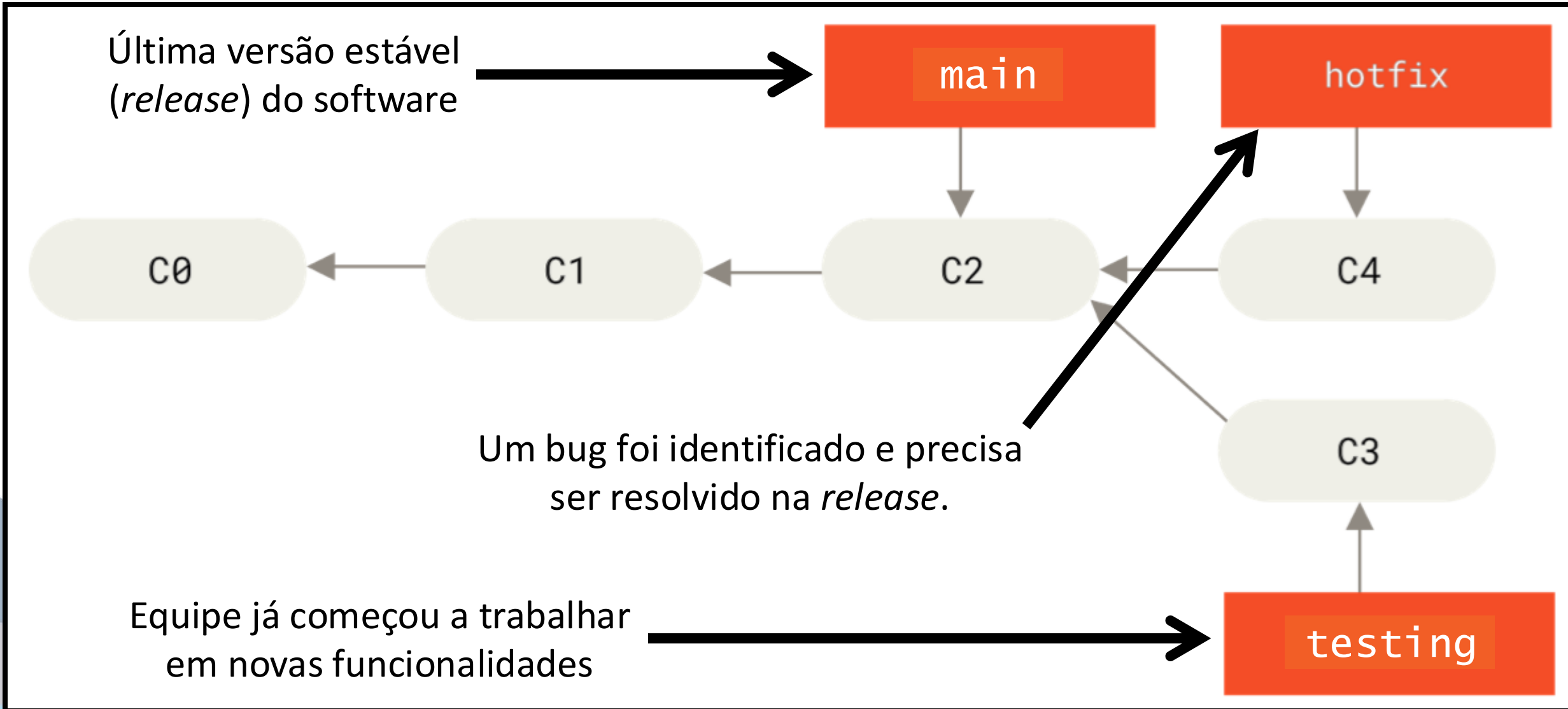
```
|/
```

```
* 1d2db7b Personaliza Hello, World! Adiciona .gitignore
```

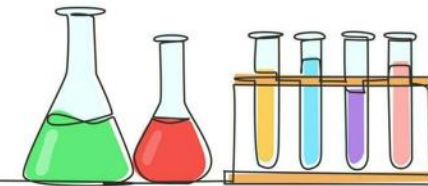
```
* a715148 Adiciona comentário à classe
```

```
* 1f3ee0e Commit inicial
```

Simulando uma situação hipotética...



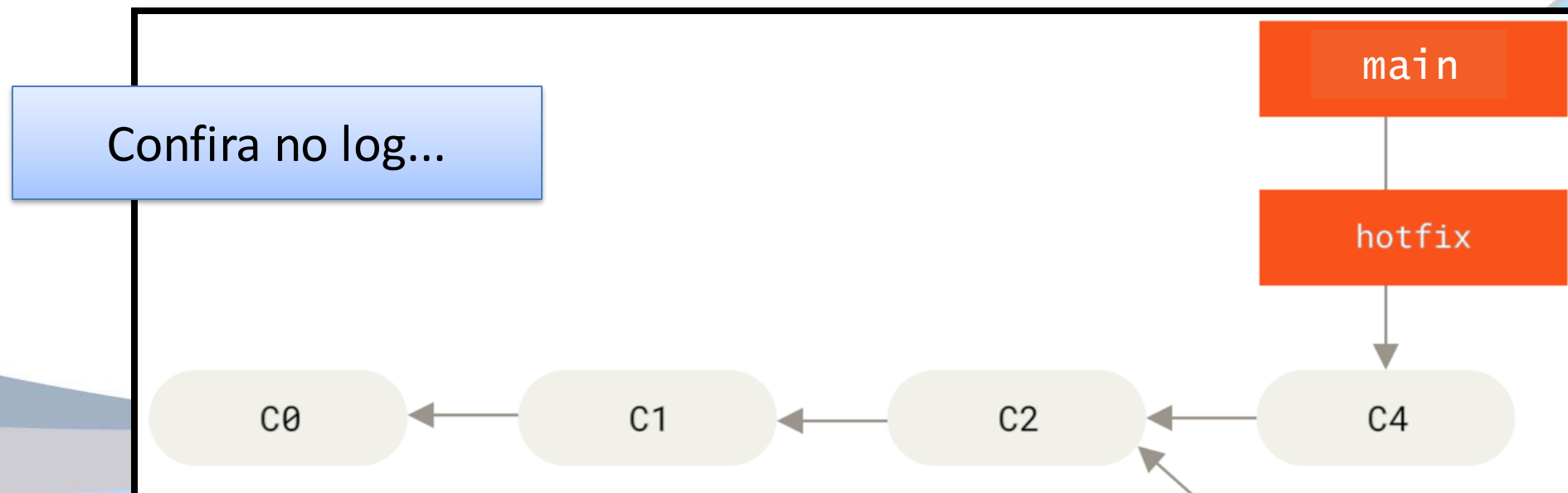
Mescle para incorporar o *hotfix*



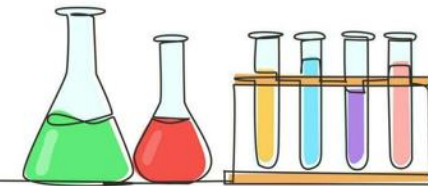
```
vitor@localhost repo % git checkout main
Switched to branch 'main'

vitor@localhost repo % git merge hotfix
Updating 3b0f42a..be33286
Fast-forward
 HelloWorld.java | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)
```

Apenas move
o ponteiro
para frente.
Sem
divergências.



Exclua ramos desnecessários



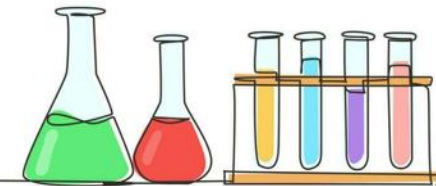
```
vitor@localhost repo % git branch -d hotfix
```

```
Deleted branch hotfix (was be33286).
```

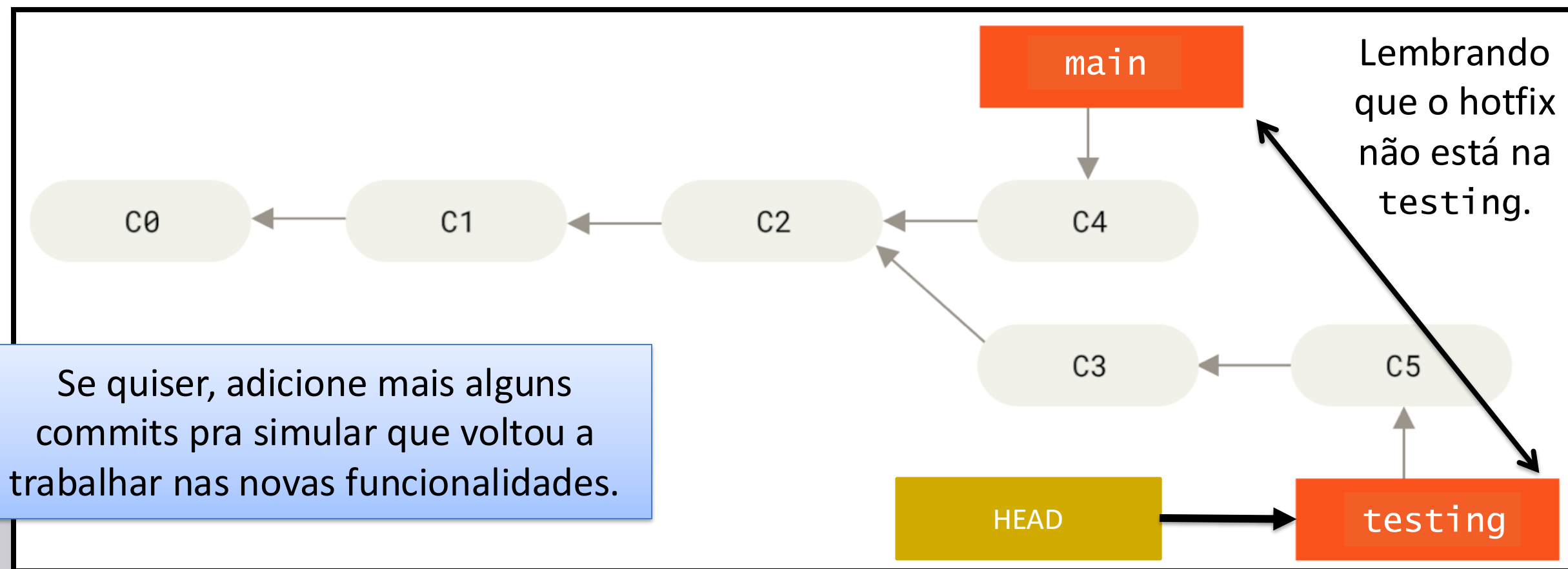
```
vitor@localhost repo % git log --oneline --decorate --graph --all
```

- * be33286 (HEAD -> main) Corrige comentário
- * 3b0f42a Comenta método main
- | * 4d9e107 (testing) Traduz para Olá, Mundo
- |/
- * 1d2db7b Personaliza Hello, World! Adiciona .gitignore
- * a715148 Adiciona comentário à classe
- * 1f3ee0e Commit inicial

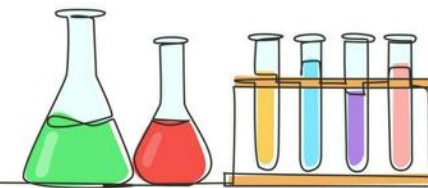
Volte ao que estava fazendo



```
vitor@localhost repo % git checkout testing
Switched to branch 'testing'
```



Mescle quando concluído



```
vitor@localhost repo % git checkout main
```

Switched to branch 'main'

```
vitor@localhost repo % git merge testing -m "Nova funcionalidade"
```

Merge made by the 'ort' strategy.

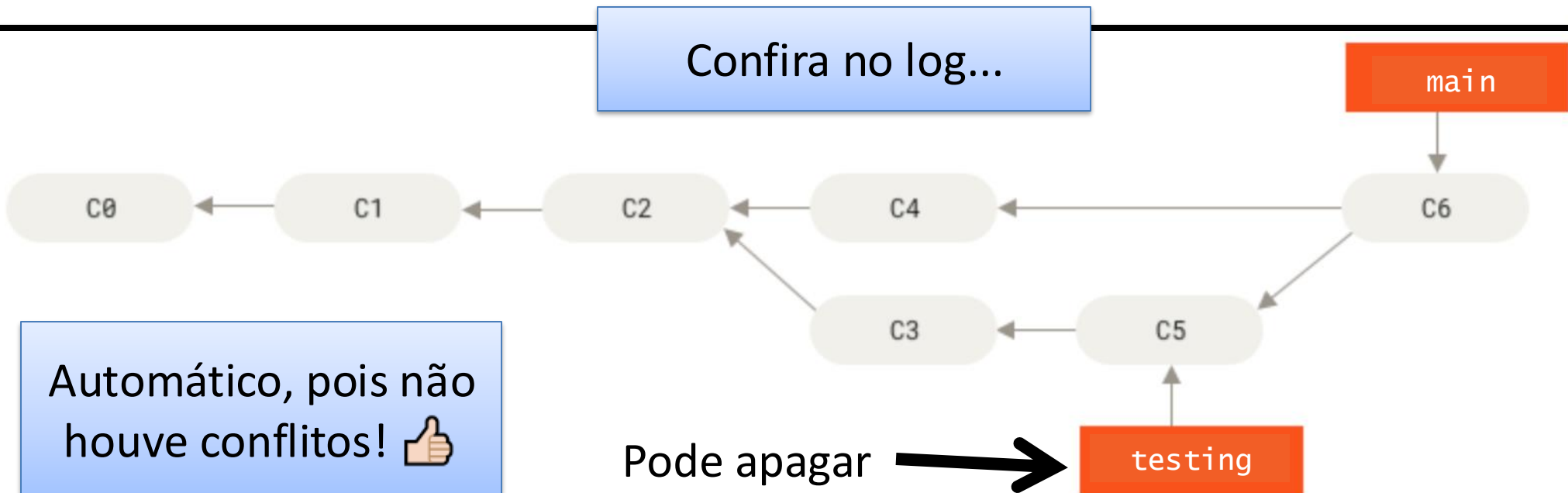
HelloWorld.java | 4 ++--

1 file changed, 2 insertions(+), 2 deletions(-)

*Ostensibly
Recursive
Three-Way.*

Mesclagem de
3 vias usando
um ancestral
comum.

Confira no log...

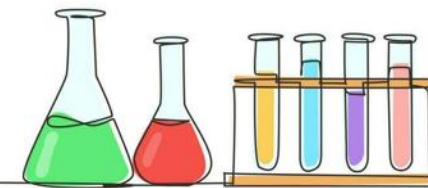


Automático, pois não
houve conflitos! 👍

Pode apagar

testing

Simule um conflito



```
git checkout -b issue12
```

main

```
git checkout main  
git checkout -b hotfix
```

issue12

Alterações nas mesmas linhas.

hotfix

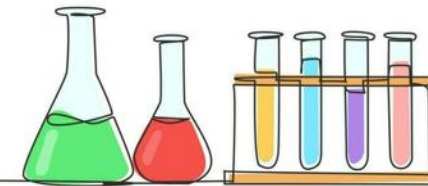
```
String who = "World/Mundo";  
if (args != null && args.length > 0)  
    who = String.join(" ", args);  
System.out.println("Hello/Olá, " + who + "!");
```

```
git commit -a -m "ptBR/enUS"
```

```
String quem = "Mundo";  
if (args != null && args.length > 0)  
    quem = String.join(" ", args);  
System.out.println("Olá, " + quem + "!");
```

```
git commit -a -m "ptBR vars"  
git checkout main  
git merge hotfix
```

Resolva o conflito manualmente



```
vitor@localhost repo % git merge issue12 -m "Merge issue #12"
```

Auto-merging HelloWorld.java

CONFLICT (content): Merge conflict in HelloWorld.java

Automatic merge failed; fix conflicts and then commit the result.

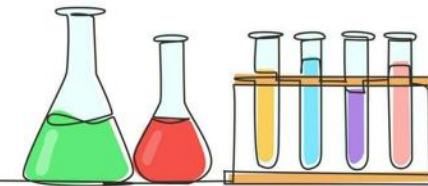
Suporte do
VSCode à
resolução de
conflitos.



```
Run | Debug
9  public static void main(String[] args) {
    Aceitar Alteração Atual | Aceitar Alteração de Entrada | Aceitar Ambas as Alterações | Comparar Alterações
10  <<<<<<< HEAD (Alteração Atual)
11      String quem = "Mundo";
12      if (args != null && args.length > 0)
13          quem = String.join(delimiter: " ", args);
14      System.out.println("Olá, " + quem + "!");
15  =====
16      String who = "World/Mundo";
17      if (args != null && args.length > 0)
18          who = String.join(delimiter: " ", args);
19      System.out.println("Hello/Olá, " + who + "!");
20  >>>>>> issue12 (Alteração da Entrada)
21  }
22  }
23
```

Resolver no Editor de Mesclagem

Confirme a resolução do conflito



```
vitor@localhost repo % git commit -a -m "Conflito resolvido"
[main c360835] Conflito resolvido
```

```
vitor@localhost repo % git log --oneline --decorate --graph --all
```

```
* c360835 (HEAD -> main) Conflito resolvido
```

```
| \
```

```
| * c43b4b4 (issue12) ptBR/enUS
```

```
* | b8b9e14 (hotfix) ptBR vars
```

```
| /
```

```
* 42f25da Nova funcionalidade
```

```
| \
```

```
| * 4d9e107 Traduz para Olá, Mundo
```

```
* | be33286 Corrige comentário
```

```
* | 3b0f42a Comenta método main
```

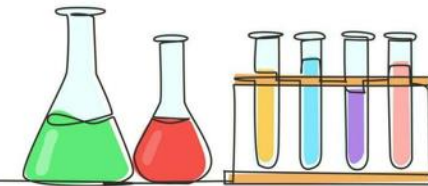
```
| /
```

```
* 1d2db7b Personaliza Hello, World! Adiciona .gitignore
```

```
* a715148 Adiciona comentário à classe
```

```
* 1f3ee0e Commit inicial
```

Veja seus atuais ramos



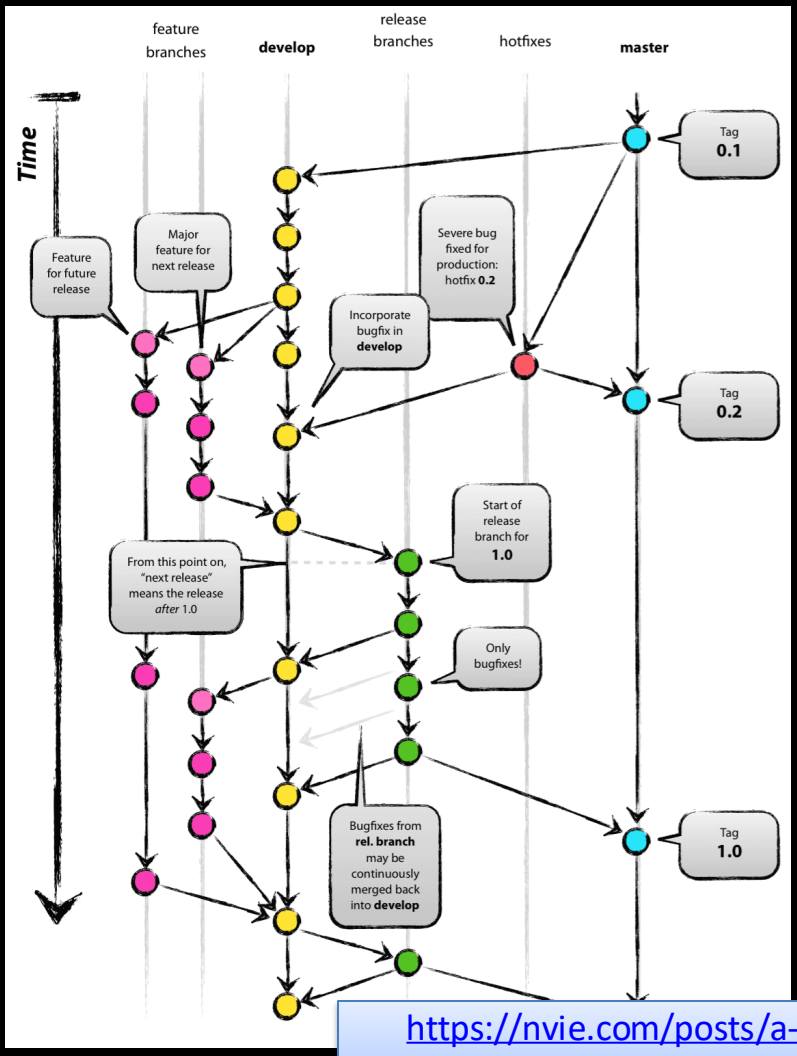
```
vitor@localhost repo % git branch
hotfix
issue12
* main
```

Indica o
HEAD

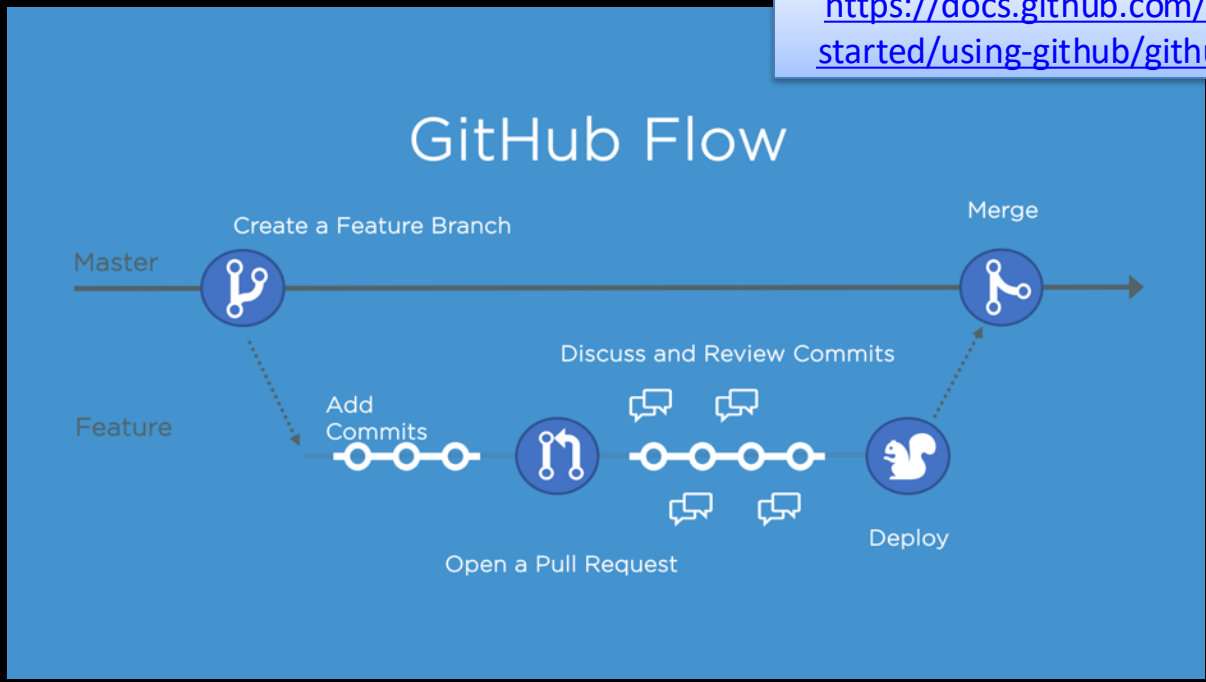
- `git branch -v`: *mostra último commit em cada ramo;*
- `git branch --merged [ramo]`: *filtra ramos mesclados no ramo indicado (default: ramo atual);*
- `git branch --no-merged [ramo]`: *filtra os não mesclados.*

Ramos não mesclados só podem ser apagados se você forçar.

Fluxo de ramos / Fluxo Git / Git Flow



<https://nvie.com/posts/a-successful-git-branching-model/>



<https://docs.github.com/pt/get-started/using-github/github-flow>

```

undefined" !=typeof b.getElementsByTagName?b.getEl
getElementsByClassName&&function(a,b){return"unde
lowcapture=''><option selected=''></option></sele
||a.push("~=")+a.querySelector(":checked").leng

```



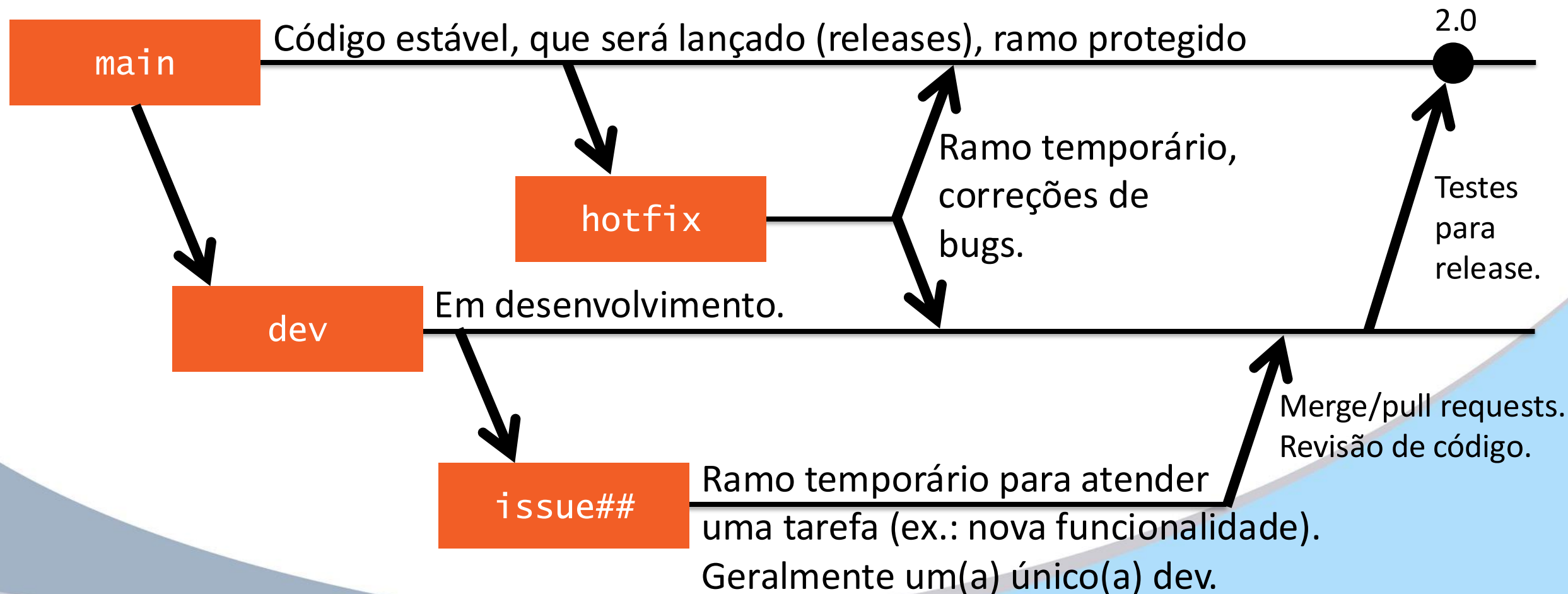
scottshipp
Posted on 9 de set. de 2019

40 ❤️ 8 🗨️

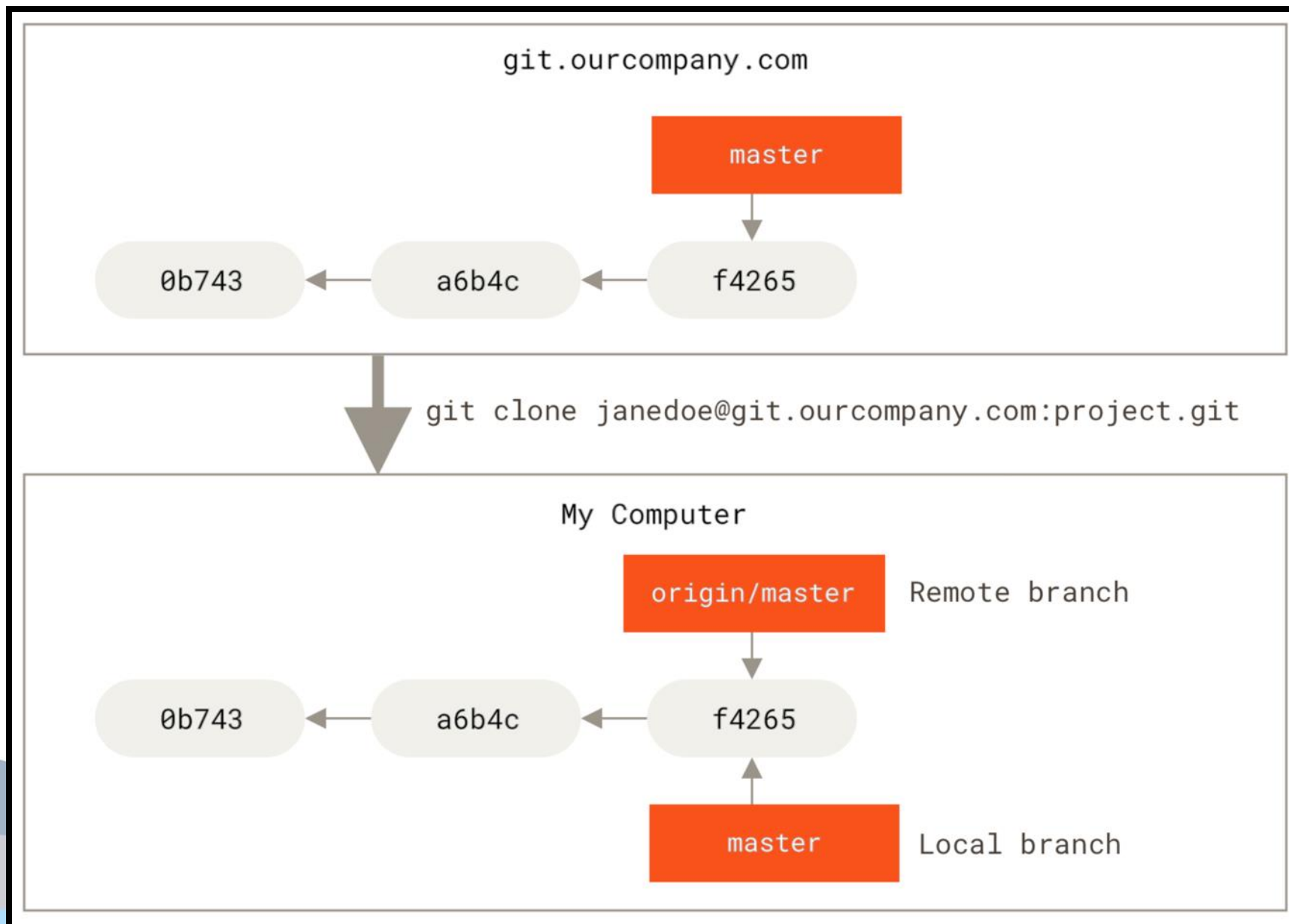
War of the Git Flows

<https://dev.to/scottshipp/war-of-the-git-flows-3ec2>

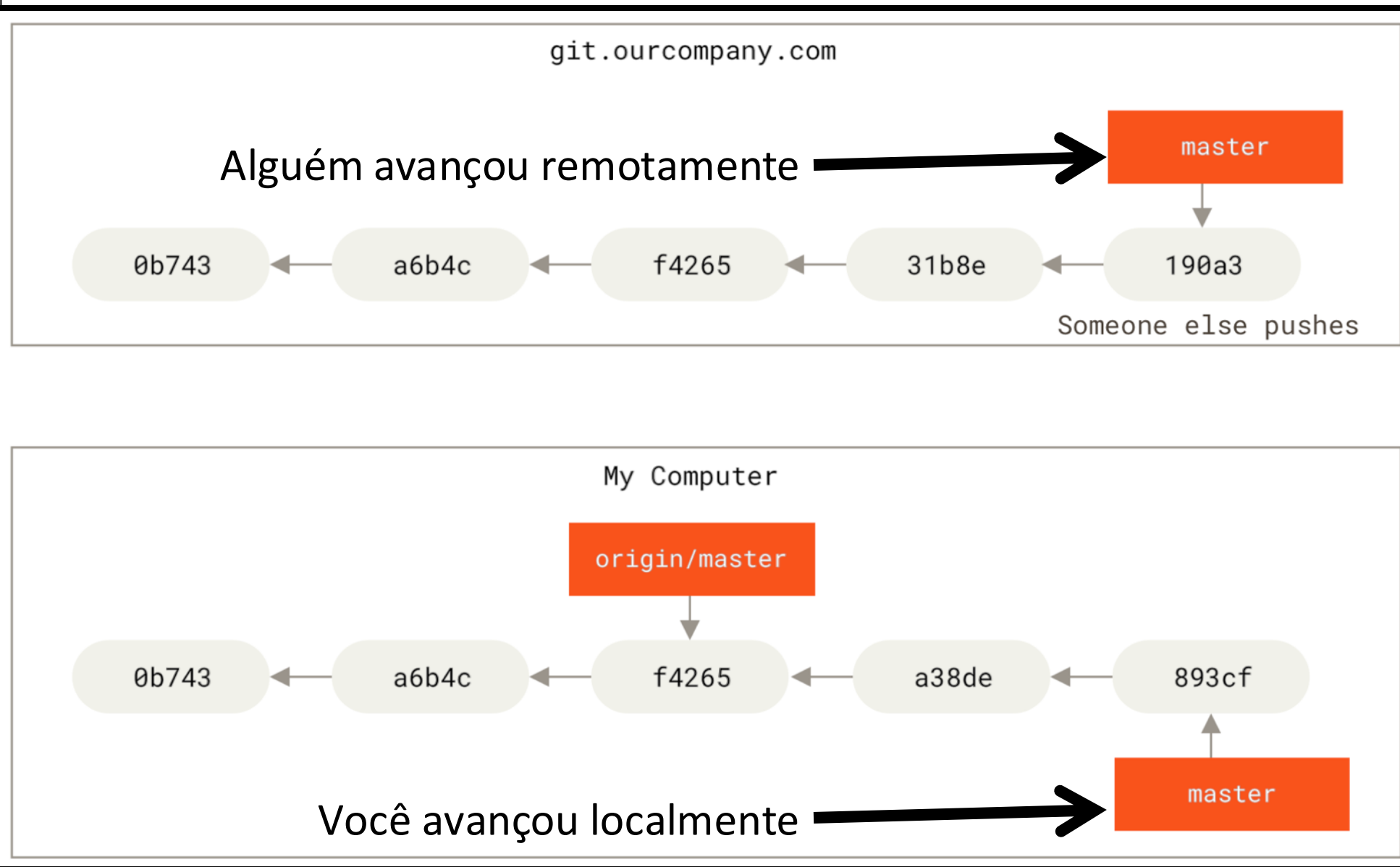
Uma proposta de fluxo “simples”



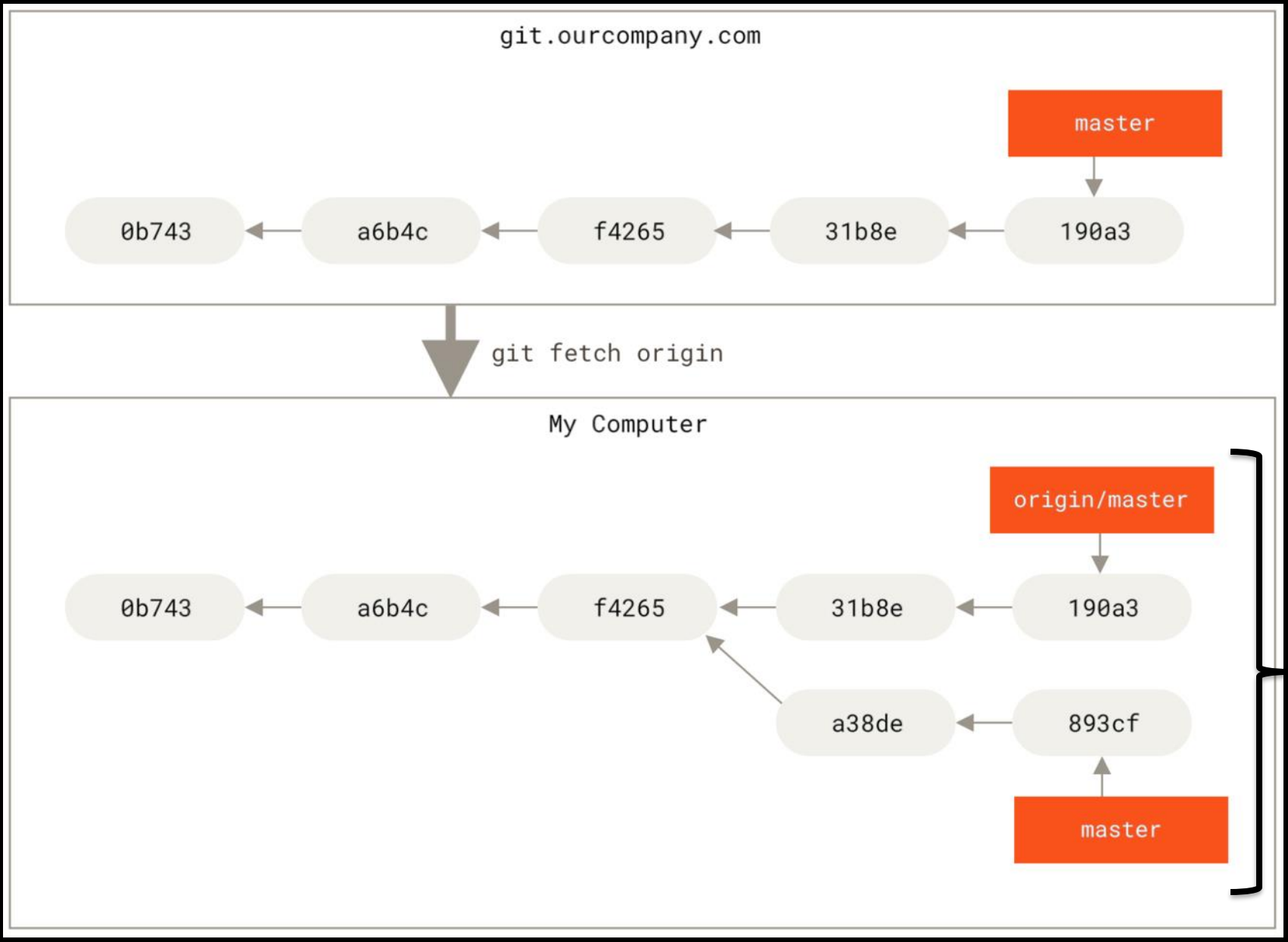
Ramos (de rastreamento) remotos



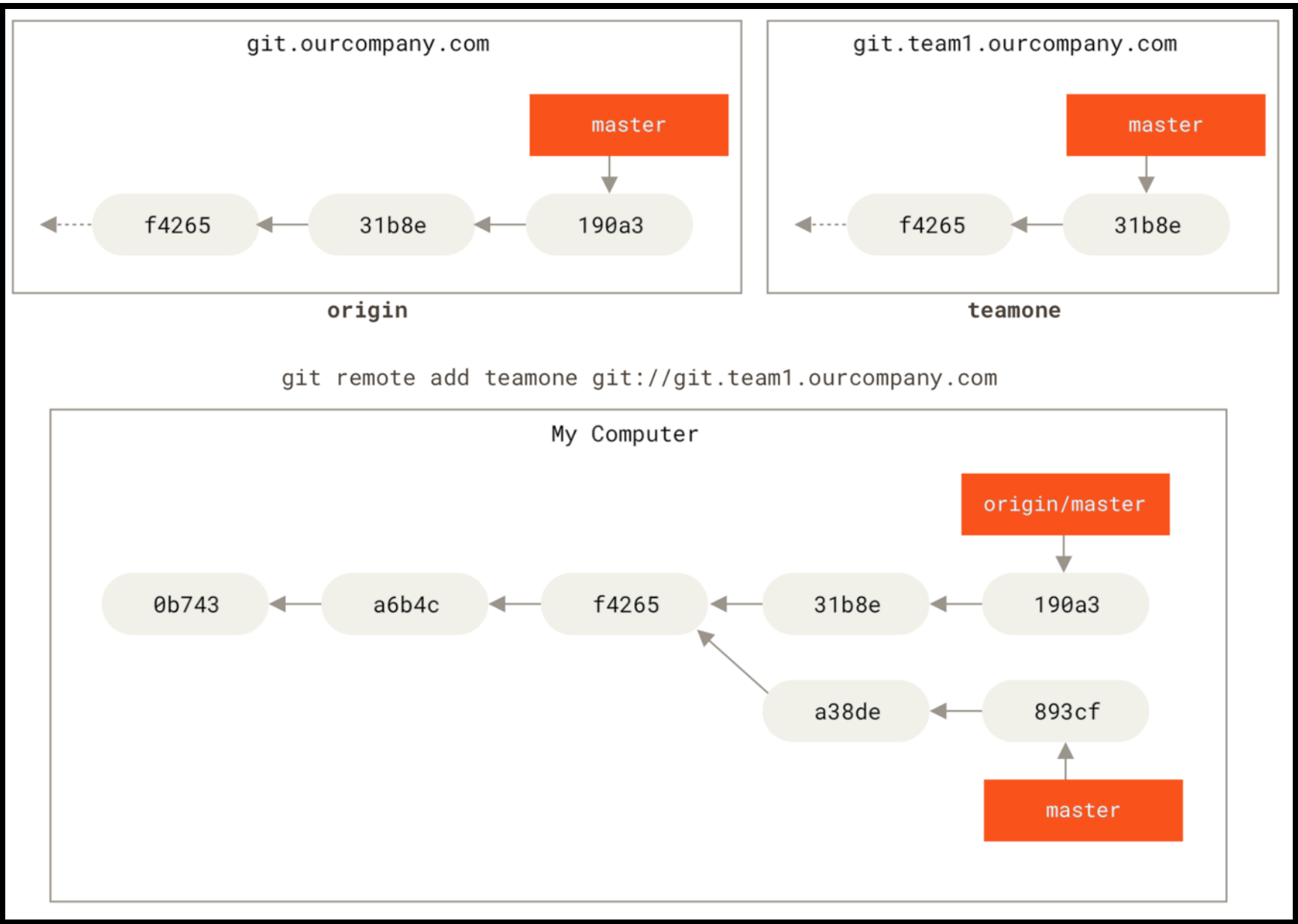
Ramos remotos: trabalho colaborativo



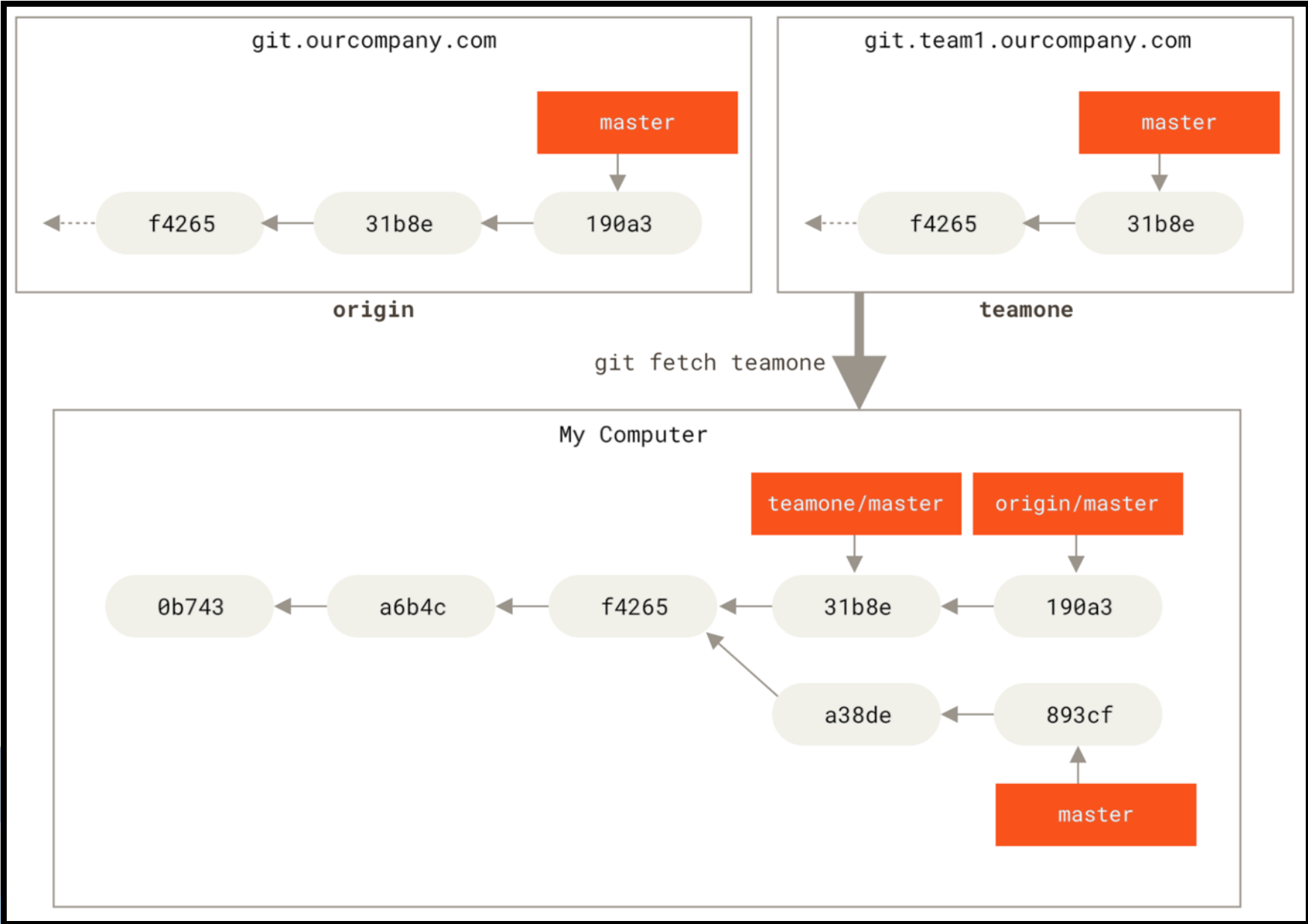
Ramos remotos: atualização



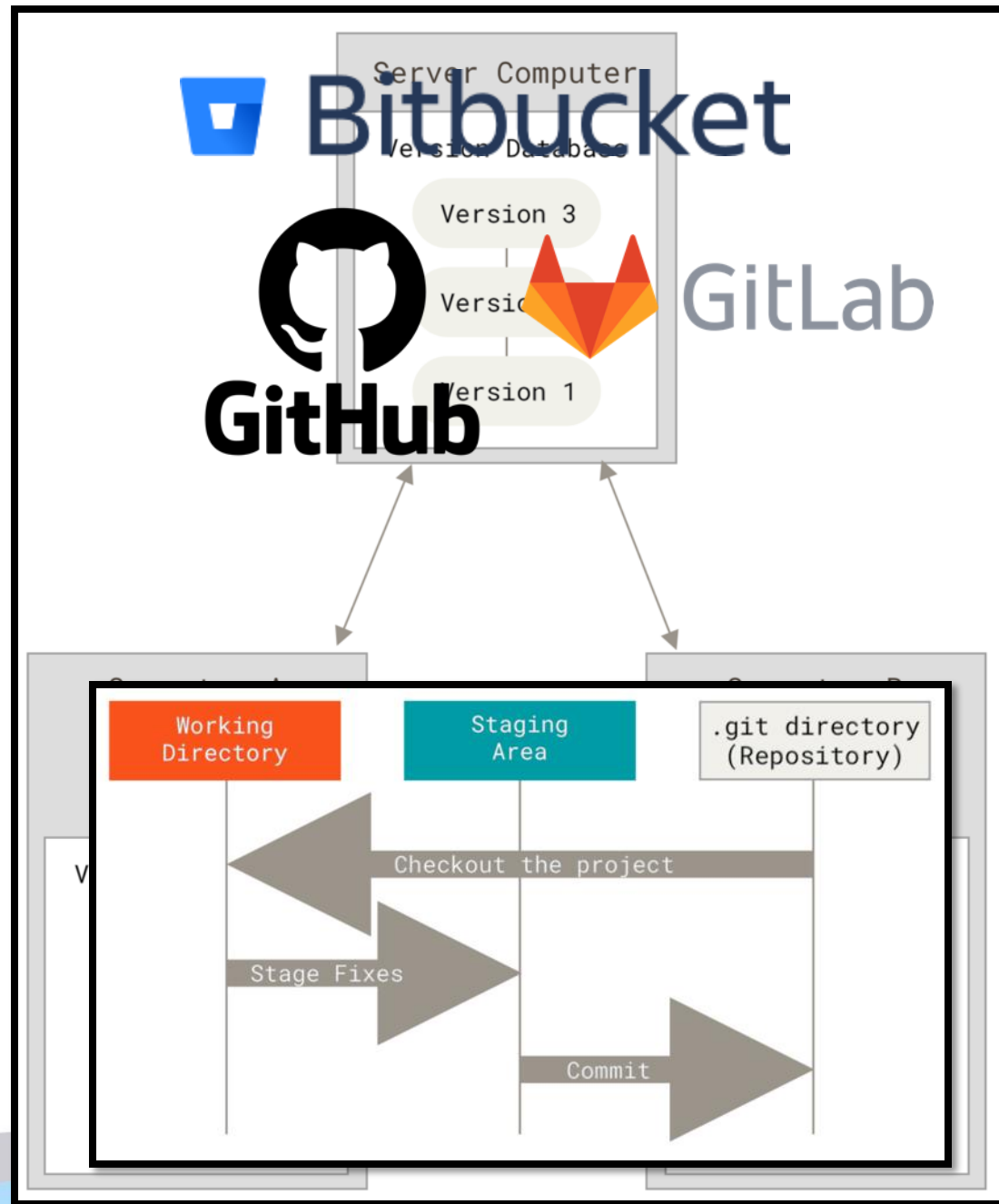
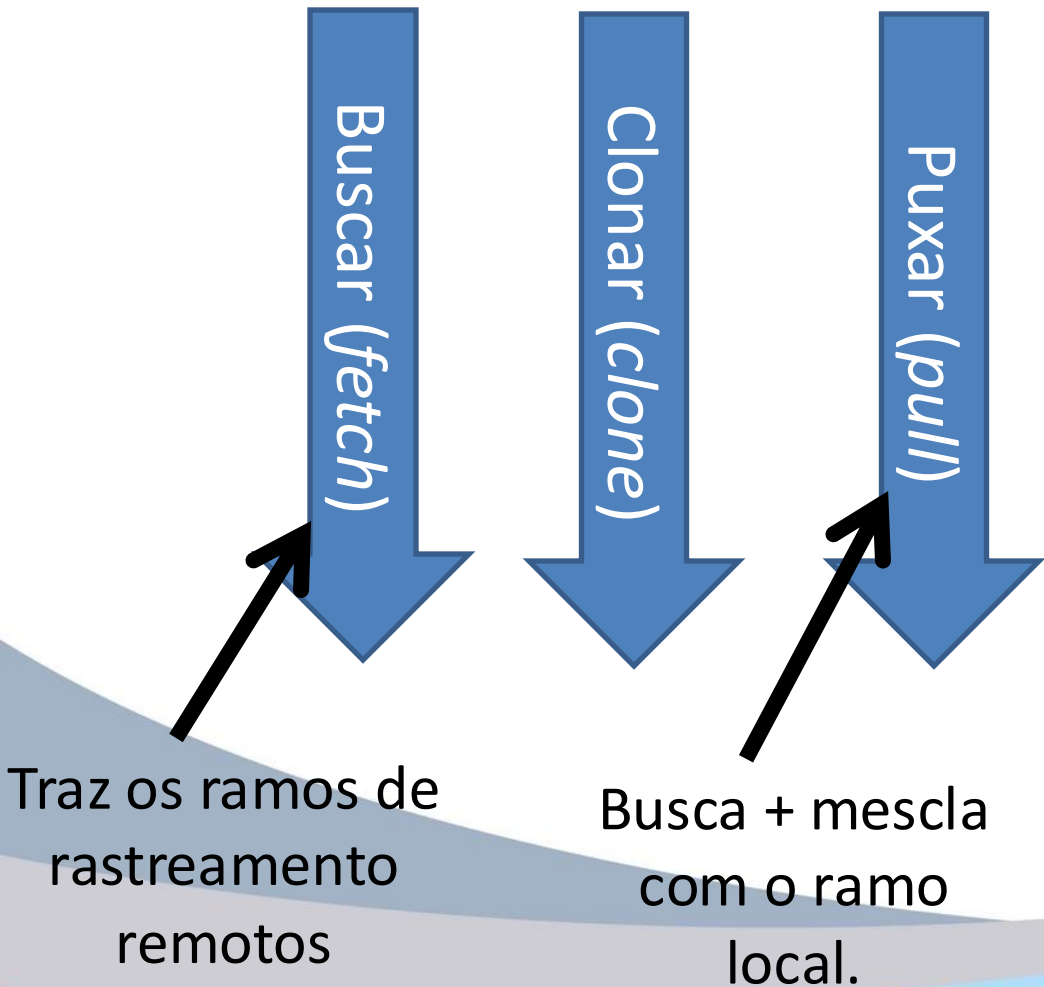
Múltiplos servidores remotos



Múltiplos ramos de rastreamento remoto



Repositórios remotos (II)



Criação de ramos locais rastreadores

- Imagine o seguinte cenário:
 1. *Colega cria ramo issue99, envia ao remoto e pede ajuda;*
 2. *Para obter o ramo de rastreamento remoto:*

git fetch origin
 3. *Para poder fazer alterações, é necessário um ramo local:*

git checkout -b issue99 origin/issue99
 4. *Você faz alterações, commit, push, etc.*

Permite o funcionamento do
`git pull` (fetch + merge)

Cria ramo local que
rastrea o ramo remoto,
ou *tracking branch*

O ramo remoto
rastreado é chamado
de *upstream branch*

Criação de ramos locais rastreadores

- Outras formas de criar *tracking branches*:

- *Um atalho:*

```
git checkout --track origin/issue99
```

- *Ou, caso não exista ramo local com mesmo nome:*

```
git checkout issue99
```

- Se o ramo local já existe e você quer que ele rastreie um ramo remoto:

```
git branch --set-upstream-to origin/issue99
```

```
git branch -u origin/issue99
```

Vendo os ramos locais rastreadores

Ramo local está 2
commits ainda não
enviados ao remoto.

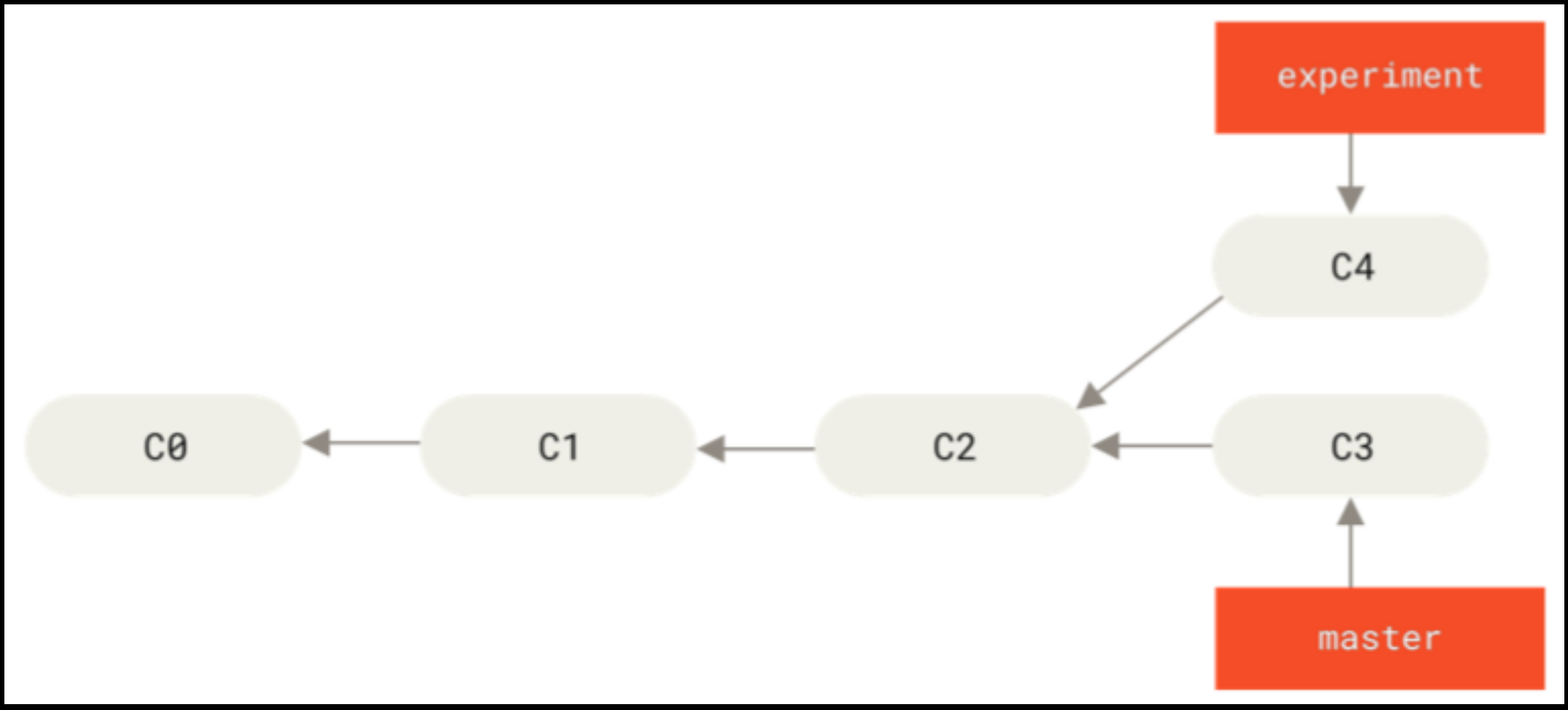
Para atualizar:
`git fetch --all`

```
$ git branch -vv
iss53      7e424c3 [origin/iss53: ahead 2] forgot the brackets
master     1ae2a45 [origin/master] deploying index fix
* serverfix f8674d9 [teamone/server-fix-good: ahead 3, behind 1] this should do it
testing    5ea463a trying something new
```

Ramos master e testing estão
atualizados em relação ao remoto.

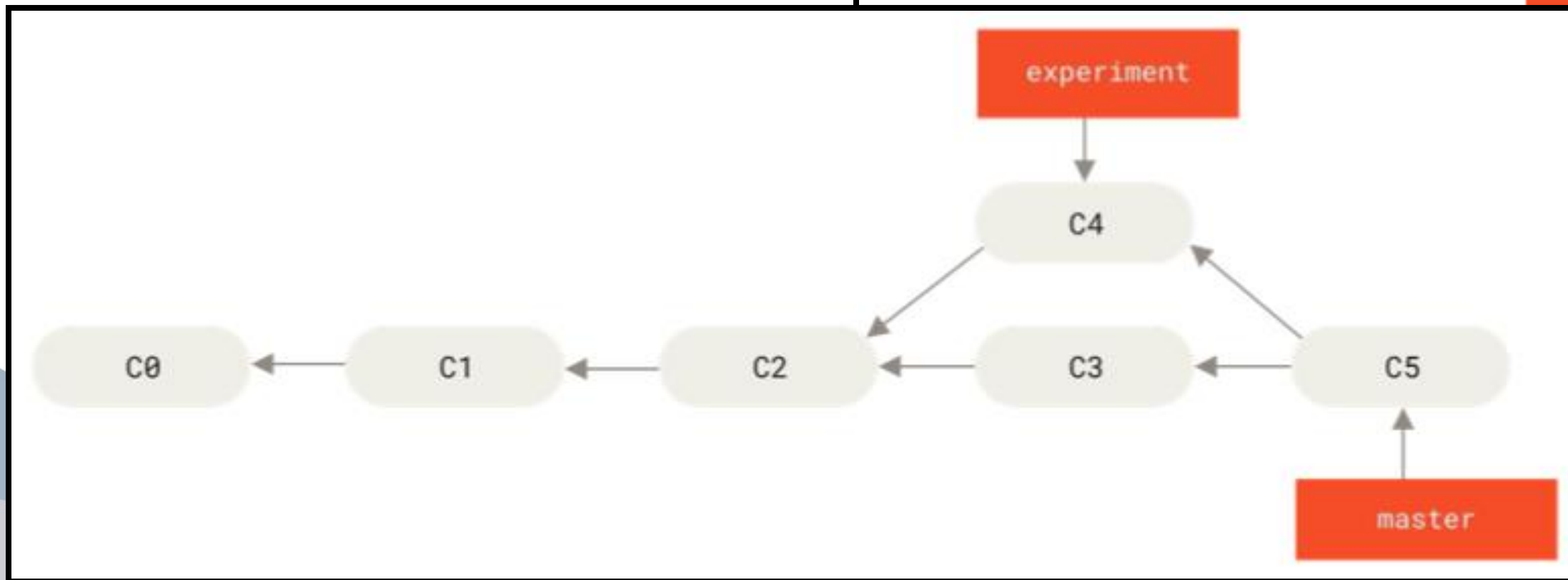
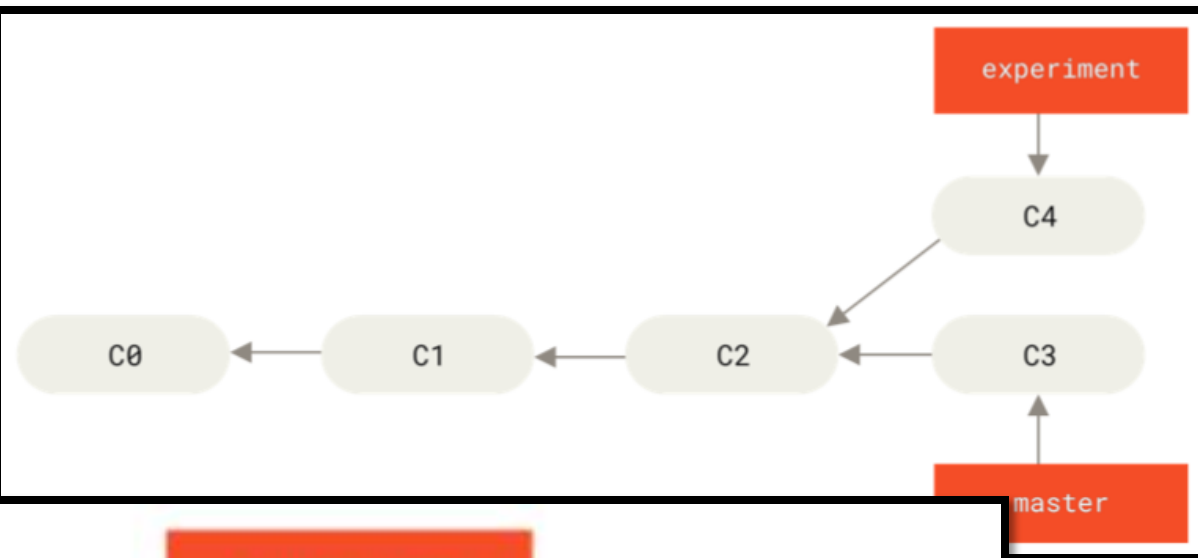
Ramo local tem 3 *commits* ainda não
enviados mas também 1 não recebido
(precisará mesclar).

Rebase vs. merge



Rebase vs. merge

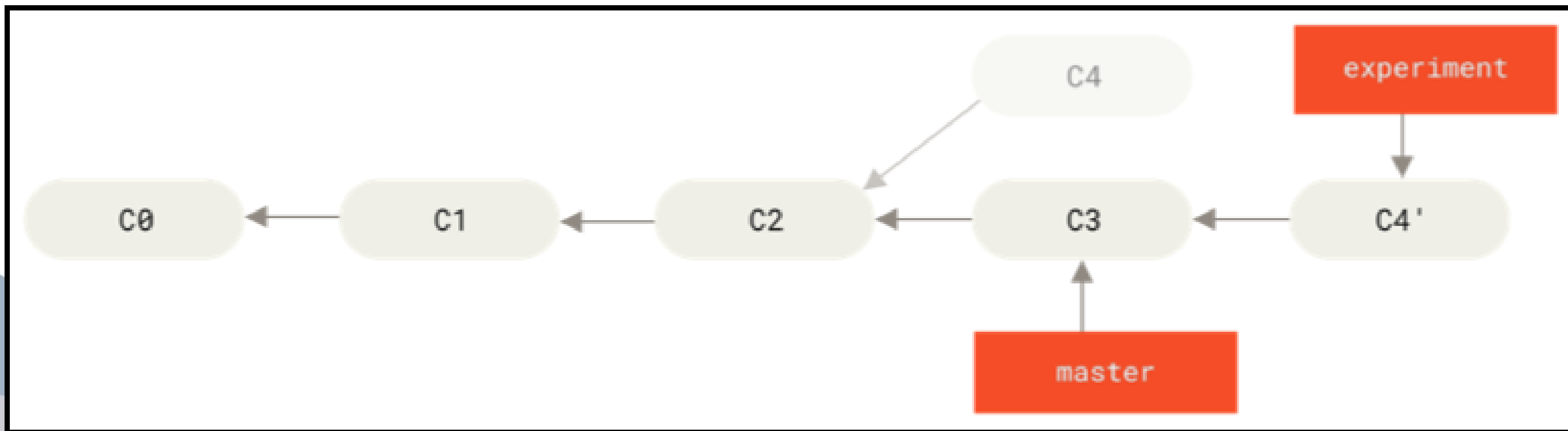
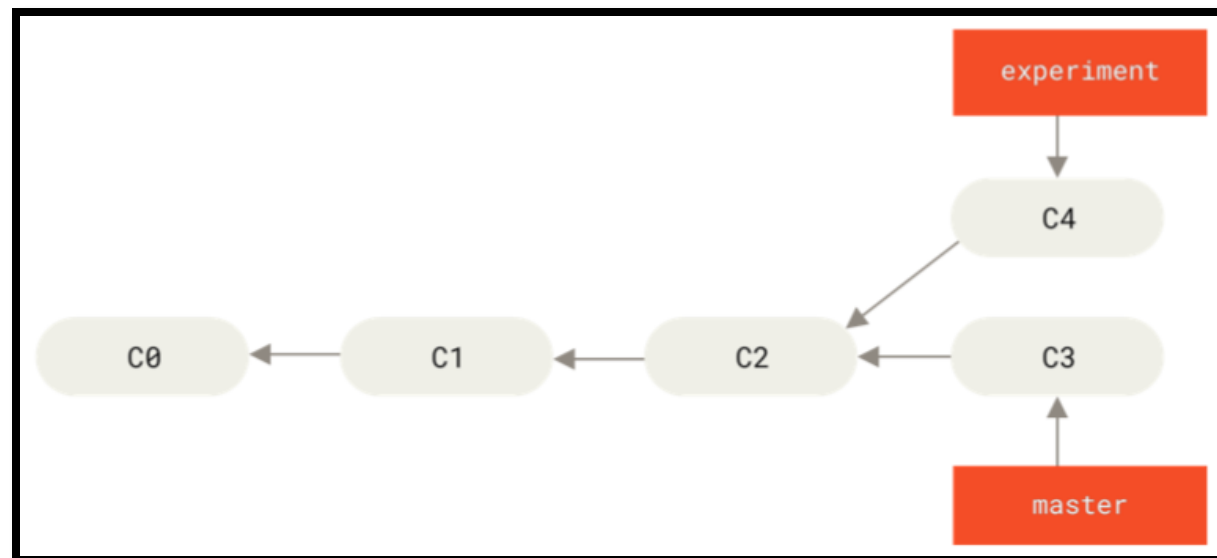
Merge



Rebase vs. merge

Rebase

(Depois faz um *merge* simples)



Rebase vs. merge

- Como o *rebase* funciona:
 1. *Vai ao ancestral comum dos dois ramos;*
 2. *Obtém o diff de cada commit do ramo em que você está;*
 3. *Aplica os diffs no ramo que está sofrendo o rebase;*
 4. *Coloca o ponteiro do ramo no resultado.*
- O *rebase* “reescreve a história”, por isso é polêmico;
- Pode ser perigoso;
 - *Dica: não faça rebase de commits que existam fora do seu repositório local (já houve push para remoto).*



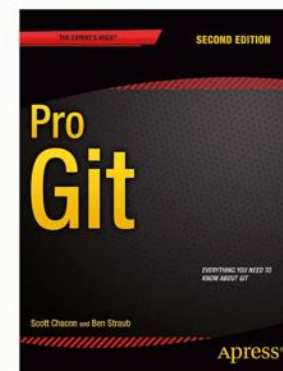
Book

The entire Pro Git book, written by Scott Chacon and Ben Straub and published by Apress, is available here. All content is licensed under the [Creative Commons Attribution Non Commercial Share Alike 3.0 license](#). Print versions of the book are available on [Amazon.com](#).

The version found here has been updated with corrections and additions from [hundreds of contributors](#). If you see an error or have a suggestion, patches and issues are welcome in its [GitHub repository](#).

1. Começando

- 1.1 Sobre Controle de Versão
- 1.2 Uma Breve História do Git
- 1.3 O Básico do Git



2nd Edition (2014)

Download Ebook



Para aprofundar:
recomendo a
leitura!

<https://git-scm.com/book/pt-br/v2>